Machine translation

Le Thanh Huong, Nguyen Kiem Hieu
School of Information and Communication Technology, HUST

An example

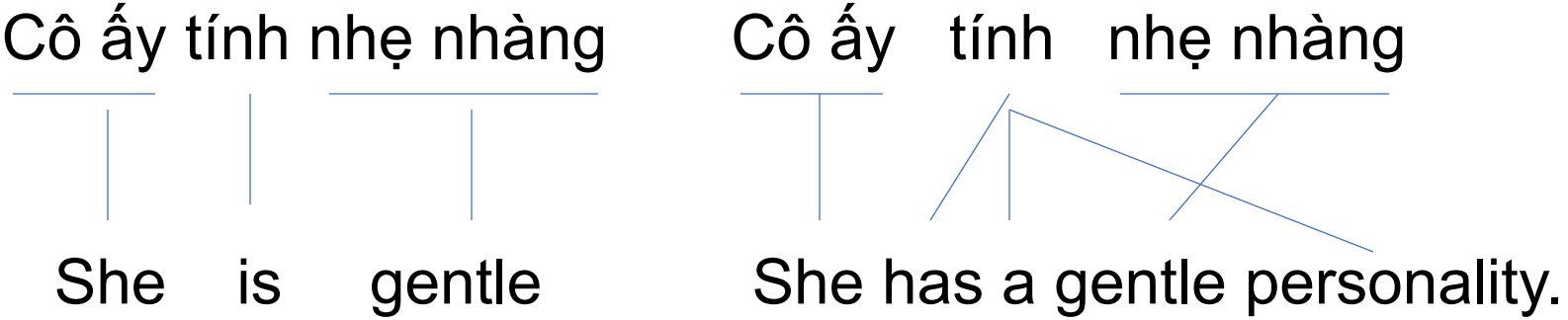
- Au sortir de la saison 97/98 et surtout au debut de cette saison 98/99…

- With leaving season 97/98 and especially at the beginning of this season 98/99…

Challenges

1. Capture variation and similarities amongst languages

- Morphologically: # morphemes/word:
  - *Isolating* languages (Vietnamese, Cantonese) – 1 word/ 1 morpheme
  - *Polysynthetic languages* (Siberian Yupik), 1 word = a whole sentence
- Degree to which morphemes are segmentable
- Example:

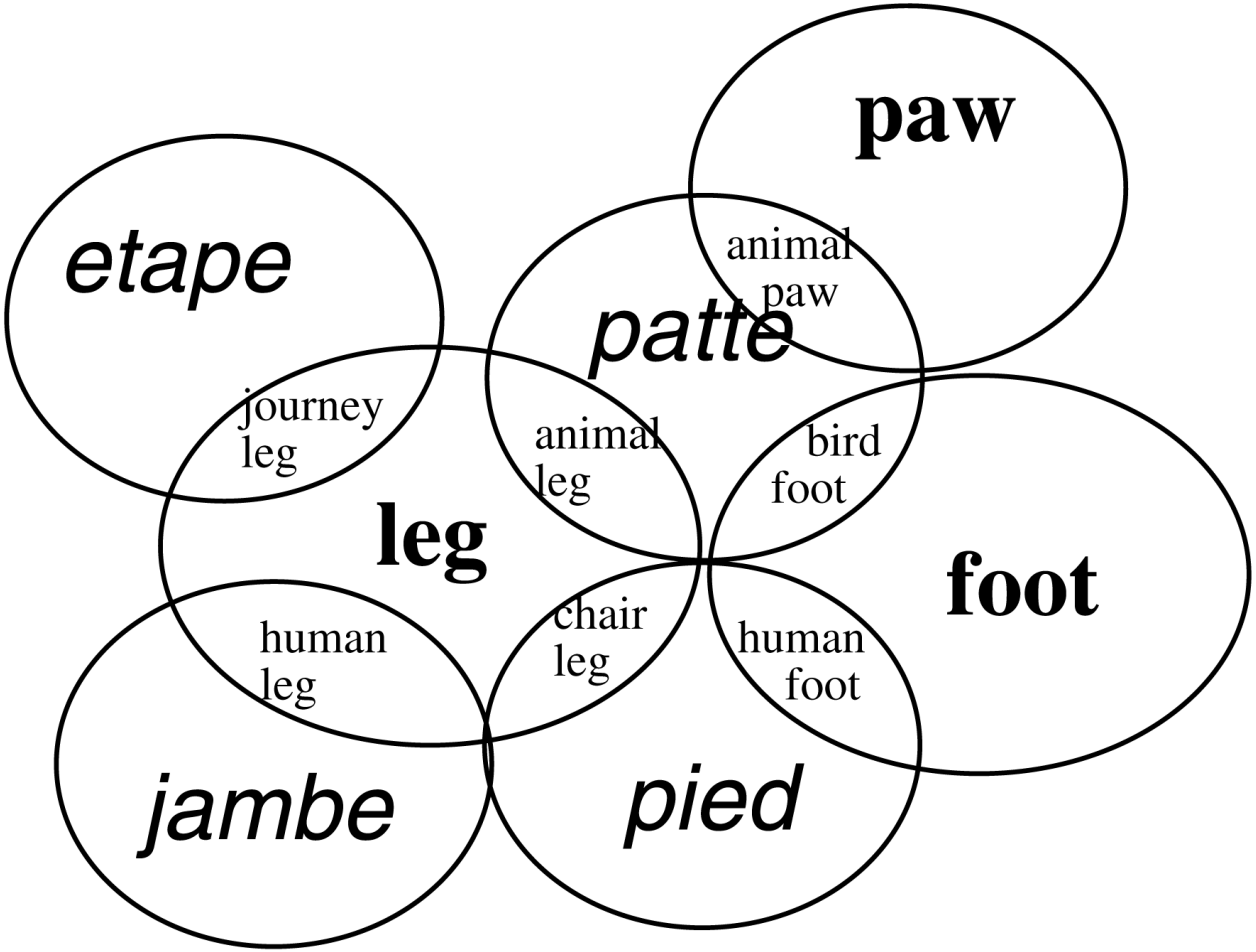


Challenges

2. Syntax: order of words in a sentence
   - *To Yukio; Yukio ne*
- English vs. Vietnamese:
   - *The* (affix1) *red* (affix2) *flag* (head)
   - *Lá cờ* (head) *đỏ* (affix2) *ấy* (affix1)
3. Differences in specificity

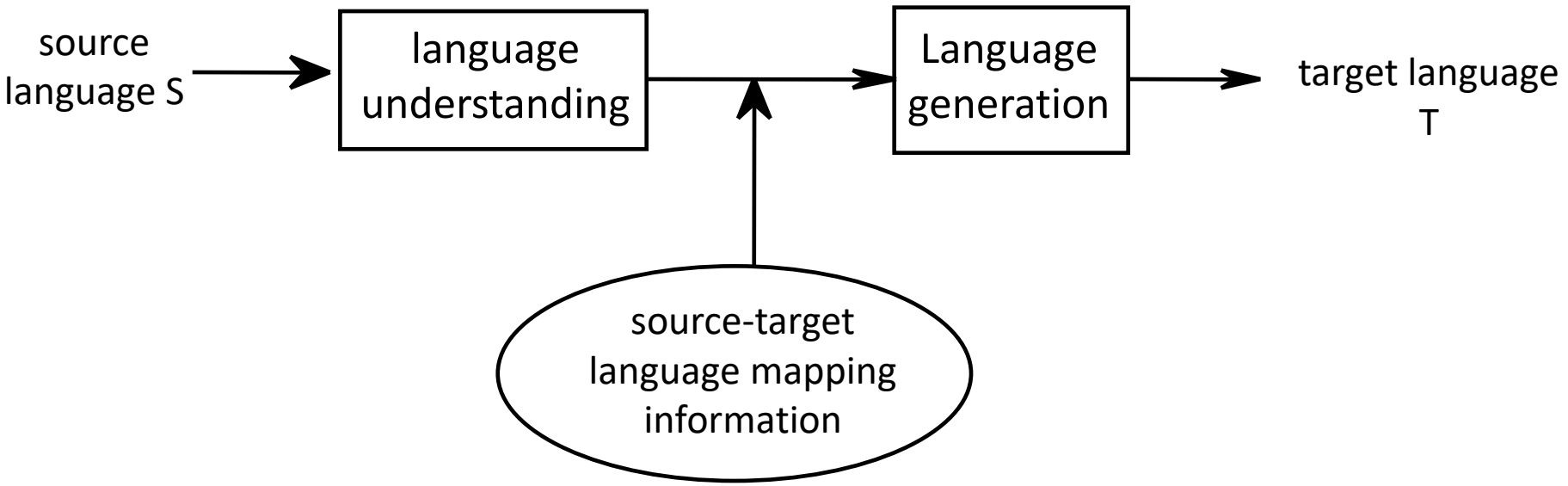
English	brother	Vietnamese	anh em
English	wall	German	wand (inside) mauer(outside)
German	berg	English	hill mountain

Conceptual space



Lexical gap: Jp, no word for *privacy;* Eng: no word for *oyakoko* (filial piety)

Three main blocks in machine translation



Language understanding

1. Lexical ambiguity:

   English: *book* - Spanish *libro, reservar*

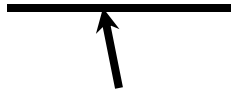
⇒ Use syntactic context

2. Syntactic ambiguity:

   I saw the guy on the hill with the telescope

3. Semantic ambiguity

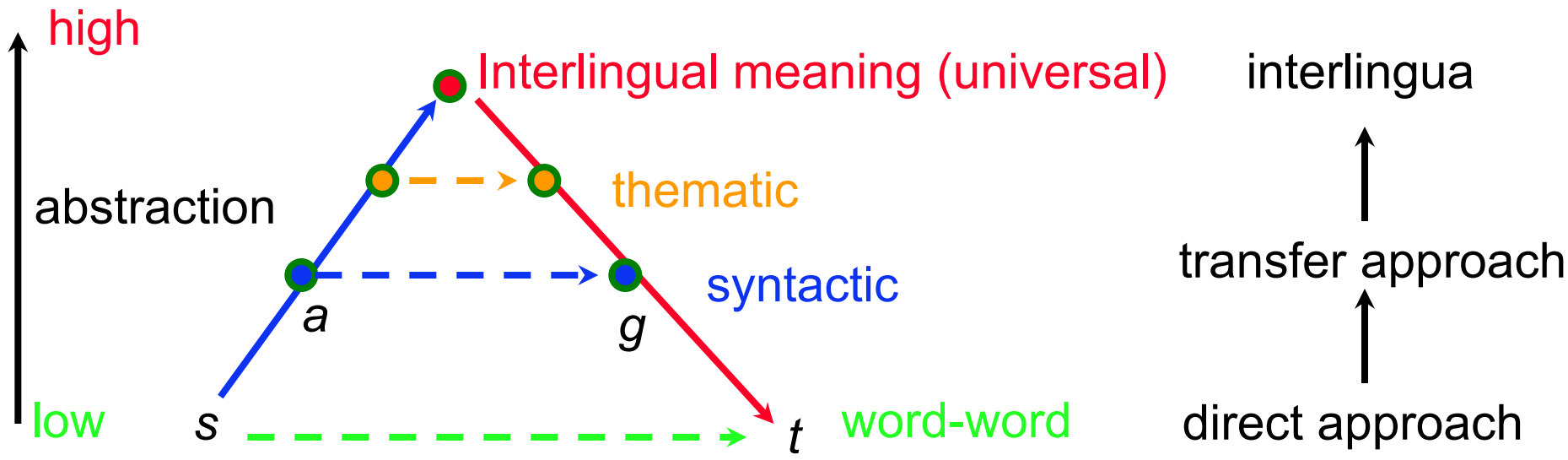
- *E: While driving, John swerved & hit a tree*



John's car

- *S: Minetras que John estaba manejando, se desvio y golpeop con un arbo*

Methods of machine translation

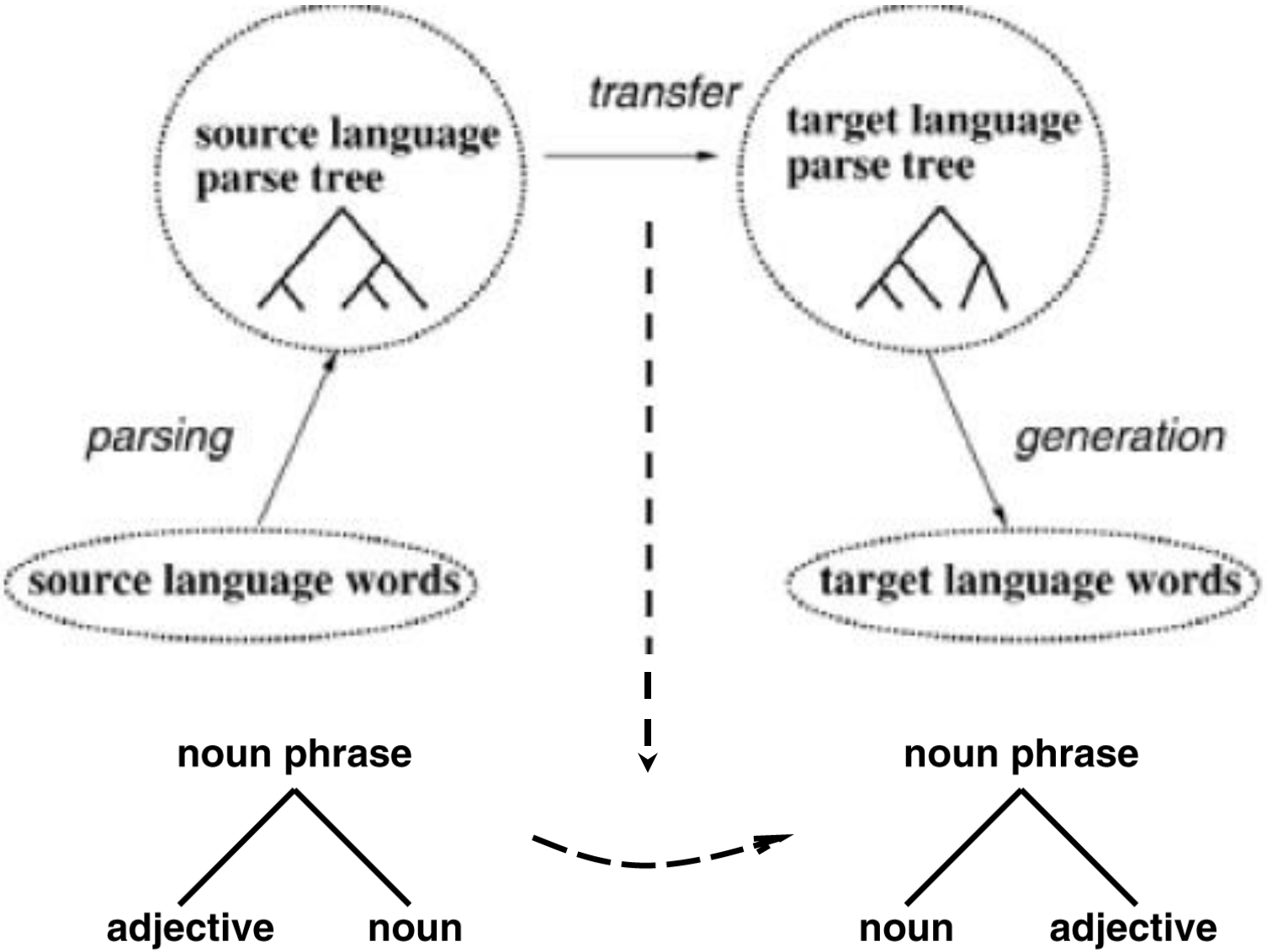


a = *a(s)*

g = *f(a(s))*; *f* - transfer function

t=g(f(a(s)))

The triangle/transfer diagram



List of transforms

English to French:

1. $\text{NP} \rightarrow \text{Adjective}_1\ \text{Noun}_2$
   $\Rightarrow$
   $\text{NP} \rightarrow \text{Noun}_2\ \text{Adjective}_1$

Japanese to English:

2. $\text{Existential-There-Sentence} \rightarrow \text{There}_1\ \text{Verb}_2\ \text{NP}_3\ \text{Postnominal}_4$
   $\Rightarrow$
   $\text{Sentence} \rightarrow (\text{NP} \rightarrow \text{NP}_3\ \text{Relative-Clause}_4)\ \text{Verb}_2$
3. $\text{NP} \rightarrow \text{NP}_1\ \text{Relative Clause}_2$
   $\Rightarrow$
   $\text{NP} \rightarrow \text{Relative-Clause}_2\ \text{NP}_1$

Interlingua approach: using meaning

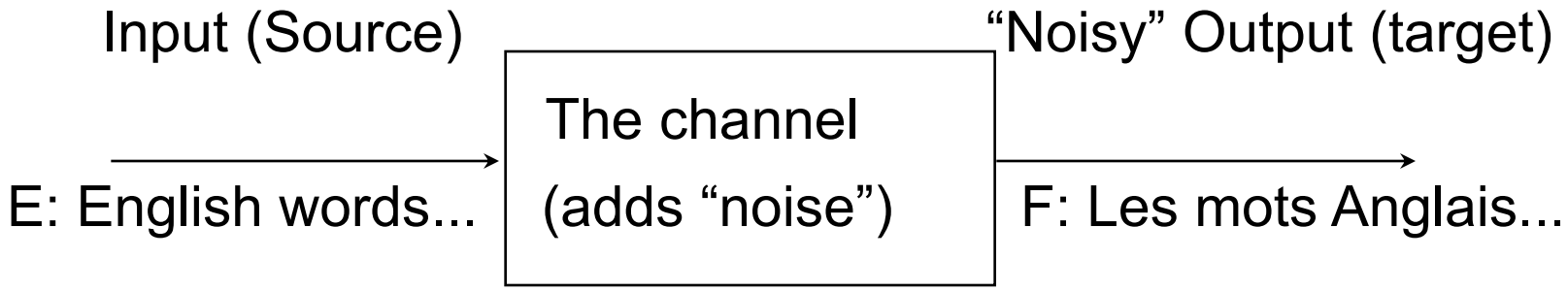
- Transfer: one pair of rules per language pair
- Objects/events (ontology)

$$
\begin{bmatrix}
\text{event} & \text{gardening} & \\
\text{agent} & \begin{bmatrix} \text{man} & \\ \text{number} & \text{sg} \\ \text{definiteness} & \text{indef} \end{bmatrix} & \\
\text{aspect} & \text{progressive} & \\
\text{tense} & \text{past} &
\end{bmatrix}
$$

Statistical machine translation

The Main Idea

- Treat translation as a noisy channel problem

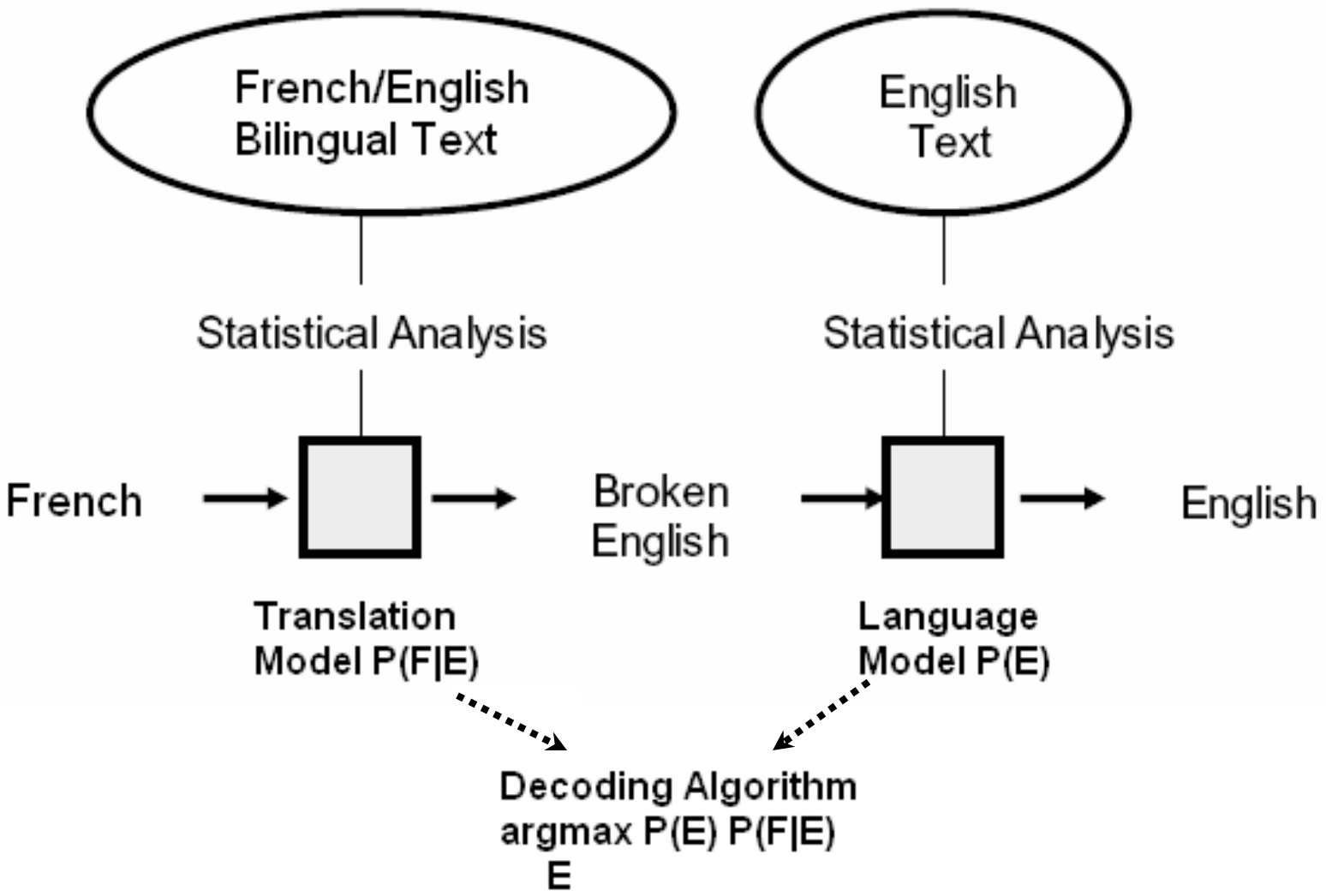


- The translation model: $P(E|F) = P(F|E)\,P(E) / P(F)$
- Interested in rediscovering E given F:

  After the usual simplification (P(F) fixed):

$$\text{argmax}_E\ P(E|F) = \text{argmax}_E\ P(F|E)\,P(E)$$

A Statistical MT System



The necessities

- Language Model (LM): our *expectation* of seeing a particular *English* (E) sentence, *a priori*:

  P(E)

- Translation Model (TM): Target sentence in *French* (F) *given* source sentence in English:

  P(F|E)

- Search procedure
  - Given F, find best E using the LM and TM distributions.
- Usual problem: sparse data!
  - We cannot create a "sentence dictionary" $E \leftrightarrow F$
  - Typically, we do not see a sentence even twice!

Alignment Idea

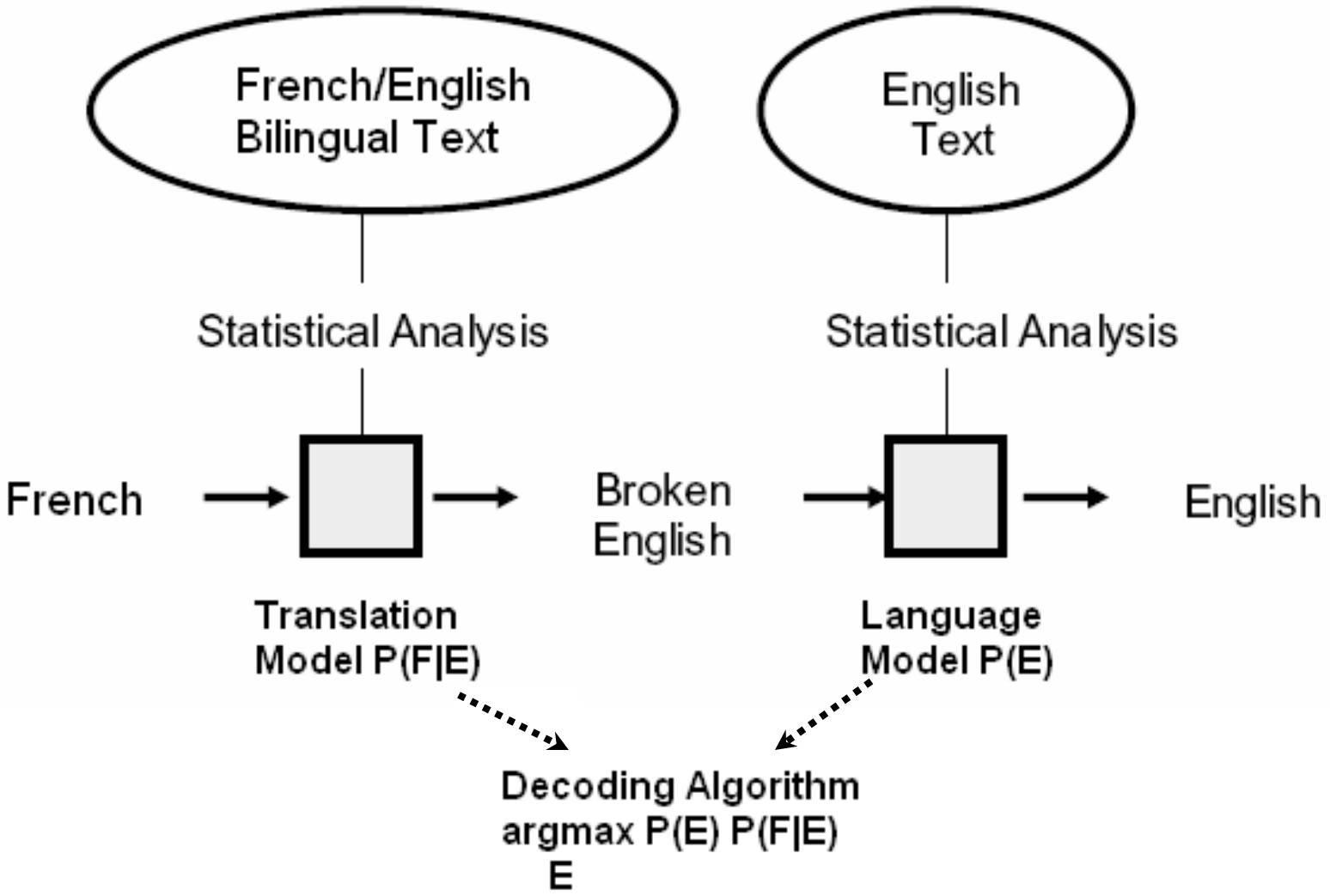
- TM doesn't care about correct strings of English words
- Use the "tagging" approach:
  - 1 English word ("tag") ~ 1 French word ("word")

  → not realistic: even #words in two sentences isn't equal

  → use "Alignment".

- Sentence alignment: find some group of sentences in one language corresponds to some other group of sentences in another language.

A Statistical MT System



The Necessities

- Language Model (LM): our *expectation* of seeing a particular *English* (E) sentence, *a priori*:

  $P(E)$

- Translation Model (TM): Target sentence in *French* (F) *given* source sentence in English:

  $P(F|E)$

- Search procedure
  - Given F, find best E using the LM and TM distributions.
- Usual problem: sparse data!
  - We cannot create a "sentence dictionary" $E \leftrightarrow F$
  - Typically, we do not see a sentence even twice!

Alignment Idea

- TM doesn't care about correct strings of English words
- Use the "tagging" approach:
  - 1 English word ("tag") ~ 1 French word ("word")

  → not realistic: even #words in two sentences isn't equal

  → use "Alignment".

- Sentence alignment: find some group of sentences in one language corresponds to some other group of sentences in another language.

Sentence Alignment

The old man is happy. He has fished many times. His wife talks to him. The fish are jumping. The sharks await.

El viejo está feliz porque ha pescado muchos veces. Su mujer habla con él. Los tiburones esperan.

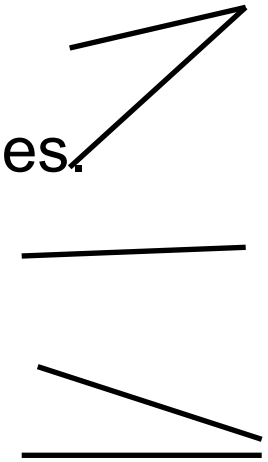
Sentence Alignment

1. The old man is happy.
2. He has fished many times.
3. His wife talks to him.
4. The fish are jumping.
5. The sharks await.

1. El viejo está feliz porque ha pescado muchos veces.
2. Su mujer habla con él.
3. Los tiburones esperan.

Sentence Alignment

1. The old man is happy.
2. He has fished many times.
3. His wife talks to him.
4. The fish are jumping.
5. The sharks await.



1. El viejo está feliz porque ha pescado muchos veces.
2. Su mujer habla con él.
3. Los tiburones esperan.

Difficulties:

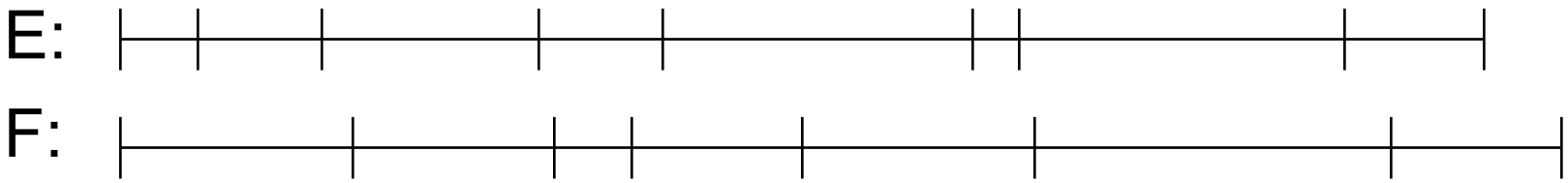
- Crossing dependencies: the order of sentences are changed in the translation.

Sentence Boundary Detection

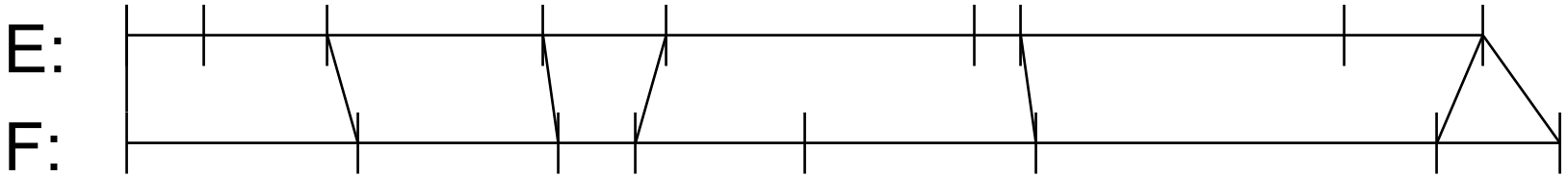
- Rules, lists:
  - Sentence breaks:
    - paragraphs (if marked)
    - certain characters: ?, !, ; (...almost sure)
    - Problem: period .
      - end of sentence (... left yesterday. He was heading to...)
      - decimal point: 3.6 (three-point-six)
      - thousand segment separator: 3.200
      - abbreviation: cf., e.g., Calif., Mt., Mr.
      - ellipsis: ...
      - other languages: ordinal number indication (2nd ~ 2.)
      - initials: A. B. Smith
- Statistical methods: e.g., Maximum Entropy

Sentence Alignment

- Problem: sentences detected only:



- **Desired output**: Segmentation with equal number of segments, spanning continuously the whole text.
- Original sentence boundaries kept:



- Alignments obtained: 2-1, 1-1, 1-1, 2-2, 2-1, 0-1

Alignment Methods

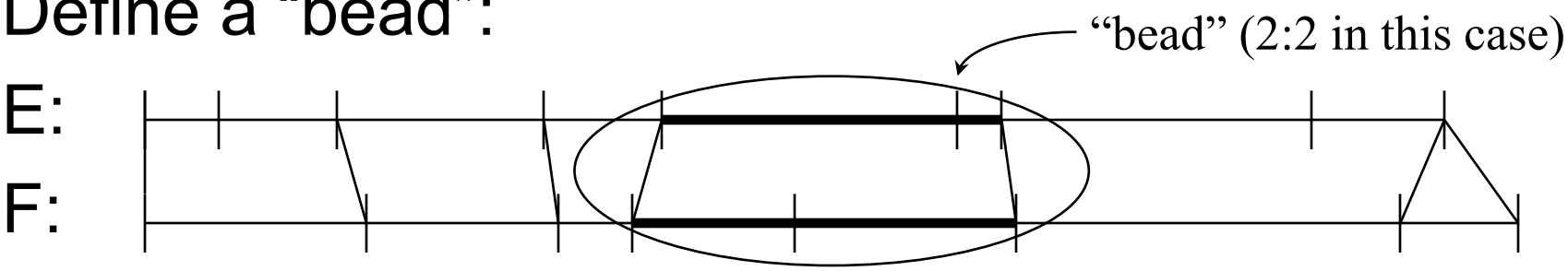
- Several methods (probabilistic and not prob.)
  - character-length based
  - word-length based
  - “cognates” (word identity used)
    - using an existing dictionary (F: prendre ~ E: make, take)
    - using word “distance” (similarity): names, numbers, borrowed words, Latin origin words, ...
- Best performing:
  - statistical, word- or character- length based (with some words perhaps)

Length-based Alignment

- First, define the problem probabilistically:

  $\text{argmax}_A P(A|E,F) = \text{argmax}_A P(A,E,F)$ (E,F fixed)

- Define a "bead":



- Approximate:

  $P(A,E,F) \cong \Pi_{i=1..n} P(B_i)$,

  where B_i is a bead; $P(B_i)$ does not depend on the rest of E,F.

The Alignment Task

Some definitions:

- Given $P(A,E,F) \cong \Pi_{i=1..n} P(B_i)$,

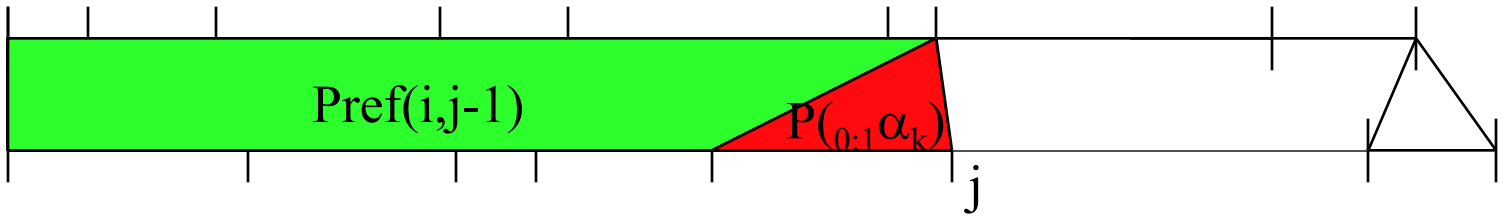
  find the partitioning of (E,F) into n beads $B_{i=1..n}$, that maximizes P(A,E,F) over training data.

- $B_i = {}_{p:q}\alpha_i$, where $p:q \in \{0:1,1:0,1:1,1:2,2:1,2:2\}$ describes the type of alignment

- Pref(i,j) - probability of the best alignment from the start of (E,F) data (1,1) up to (i,j)

Recursive Definition

- Initialize: Pref(0,0) = 0.
- Pref(i,j) = max (
  Pref(i,j-1) $P(_{0:1}\alpha_k)$, Pref(i-1,j) $P(_{1:0}\alpha_k)$, Pref(i-1,j-1) $P(_{1:1}\alpha_k)$,
  Pref(i-1,j-2) $P(_{1:2}\alpha_k)$, Pref(i-2,j-1) $P(_{2:1}\alpha_k)$, Pref(i-2,j-2) $P(_{2:2}\alpha_k)$)
- E:
- F:



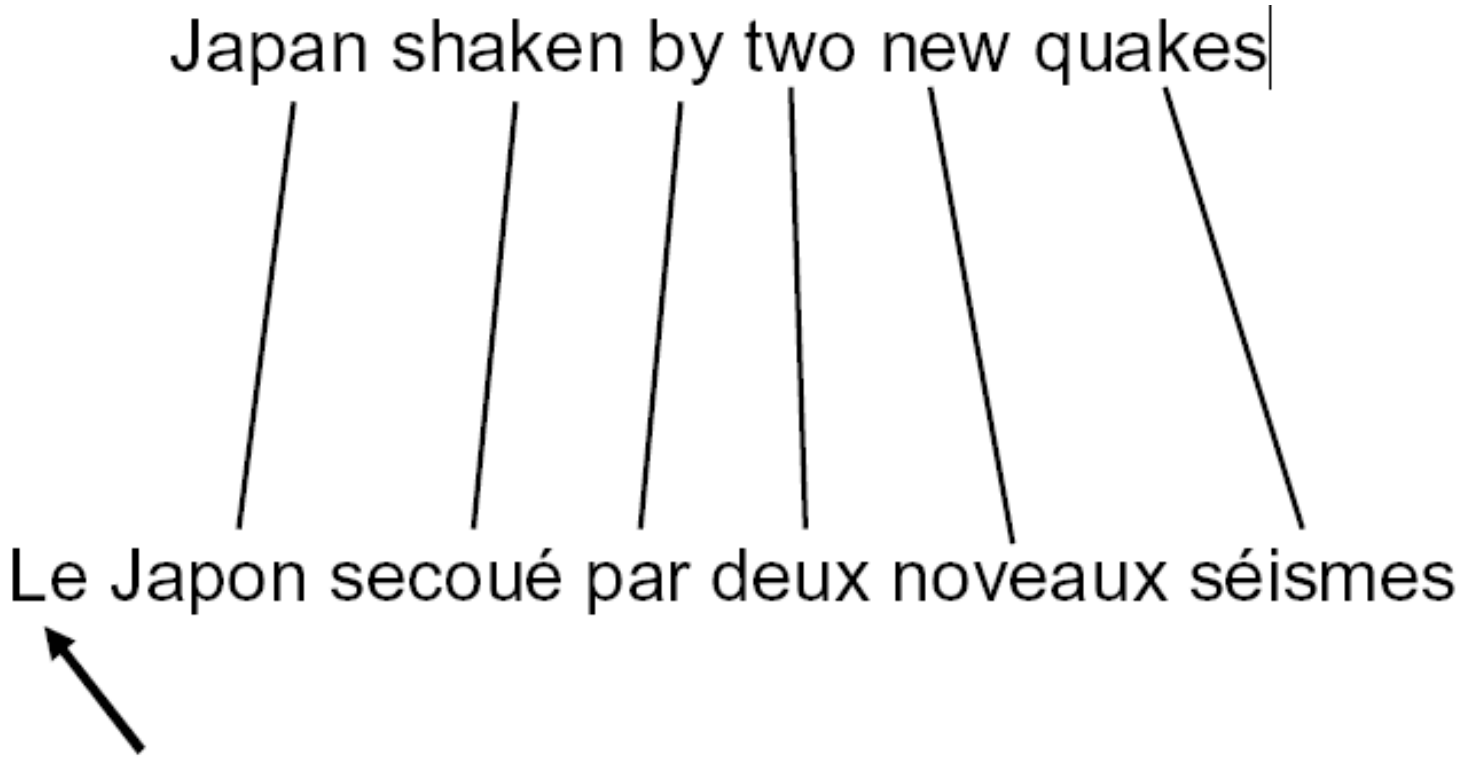
Probability of a Bead

- Remains to define $P(_{p:q}\alpha_k)$ (the red part):
  - k refers to the "next" bead, with segments of p and q sentences, lengths $l_{k,e}$ and $l_{k,f}$.
- Use normal distribution for length variation:

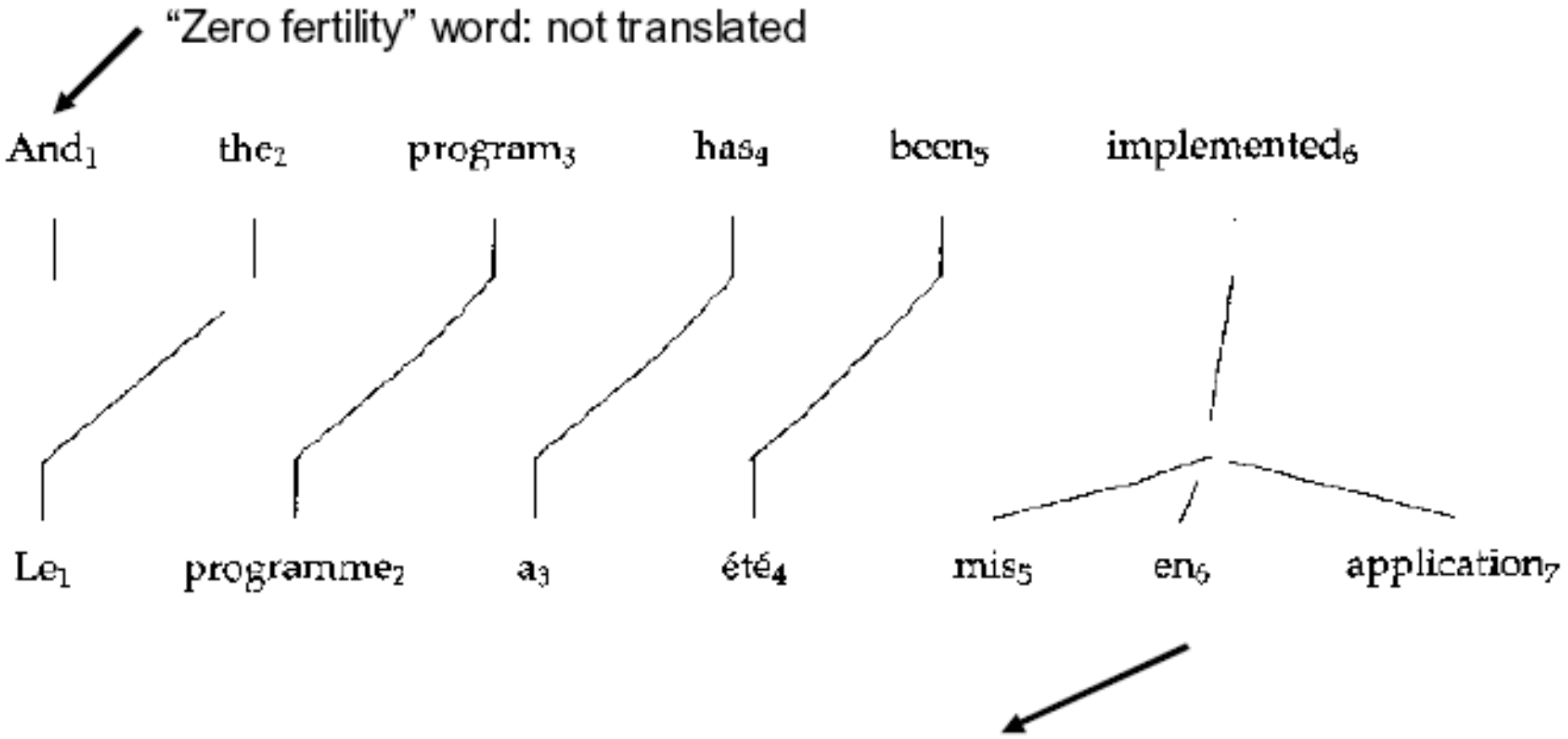
  $P(_{p:q}\alpha_k) = P(\delta(l_{k,e},l_{k,f},\mu,\sigma^2),p:q) \cong P(\delta(l_{k,e},l_{k,f},\mu,\sigma^2))P(p:q)$

  $\delta(l_{k,e},l_{k,f},\mu,\sigma^2) = (l_{k,f} - \mu l_{k,e})/\sqrt{l_{k,e}\sigma^2}$
- Estimate P(p:q) from small amount of data, or even guess and re-estimate after aligning some data.
- Words etc. might be used as better clues in $P(_{p:q}\alpha_k)$.

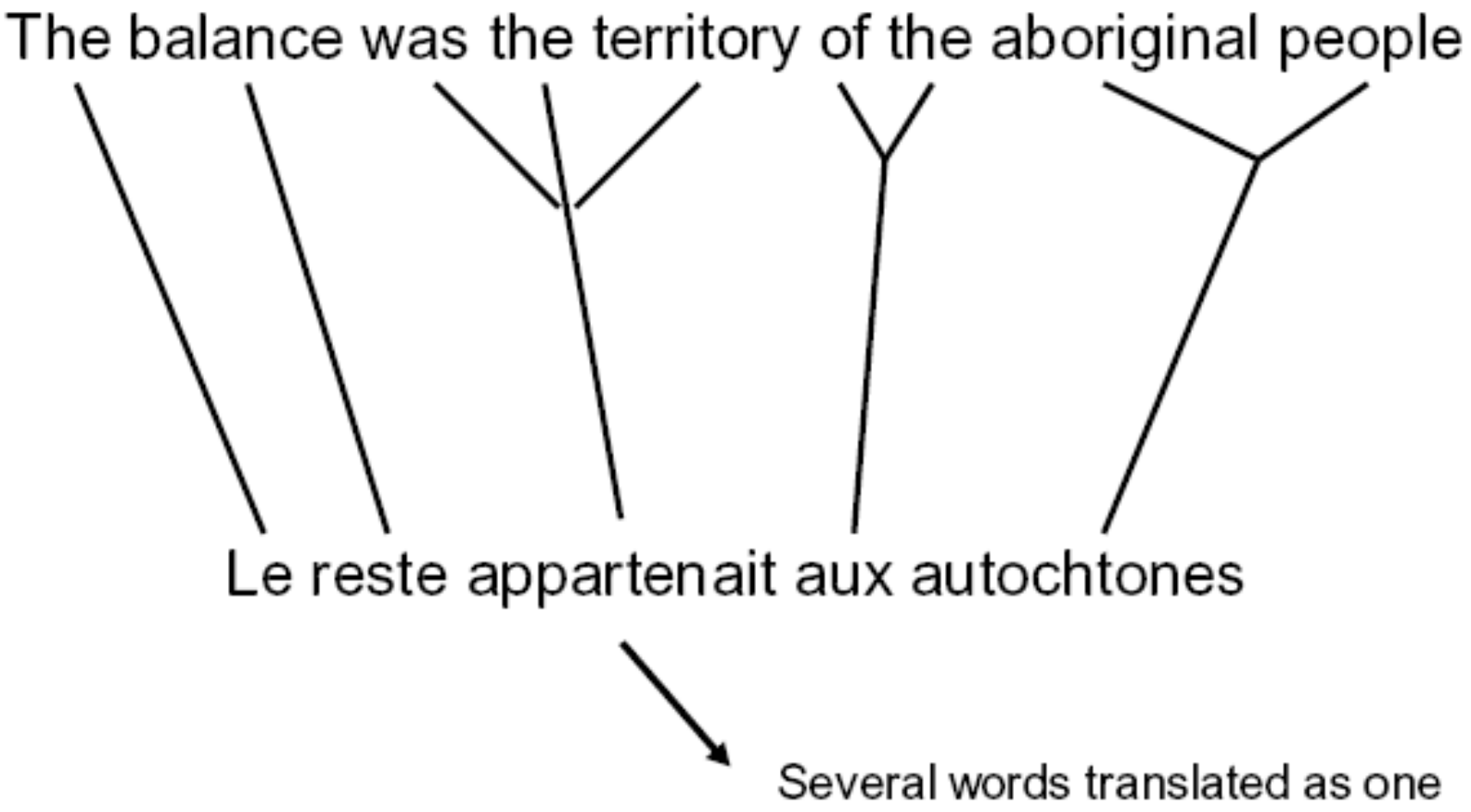
Word alignment - easy



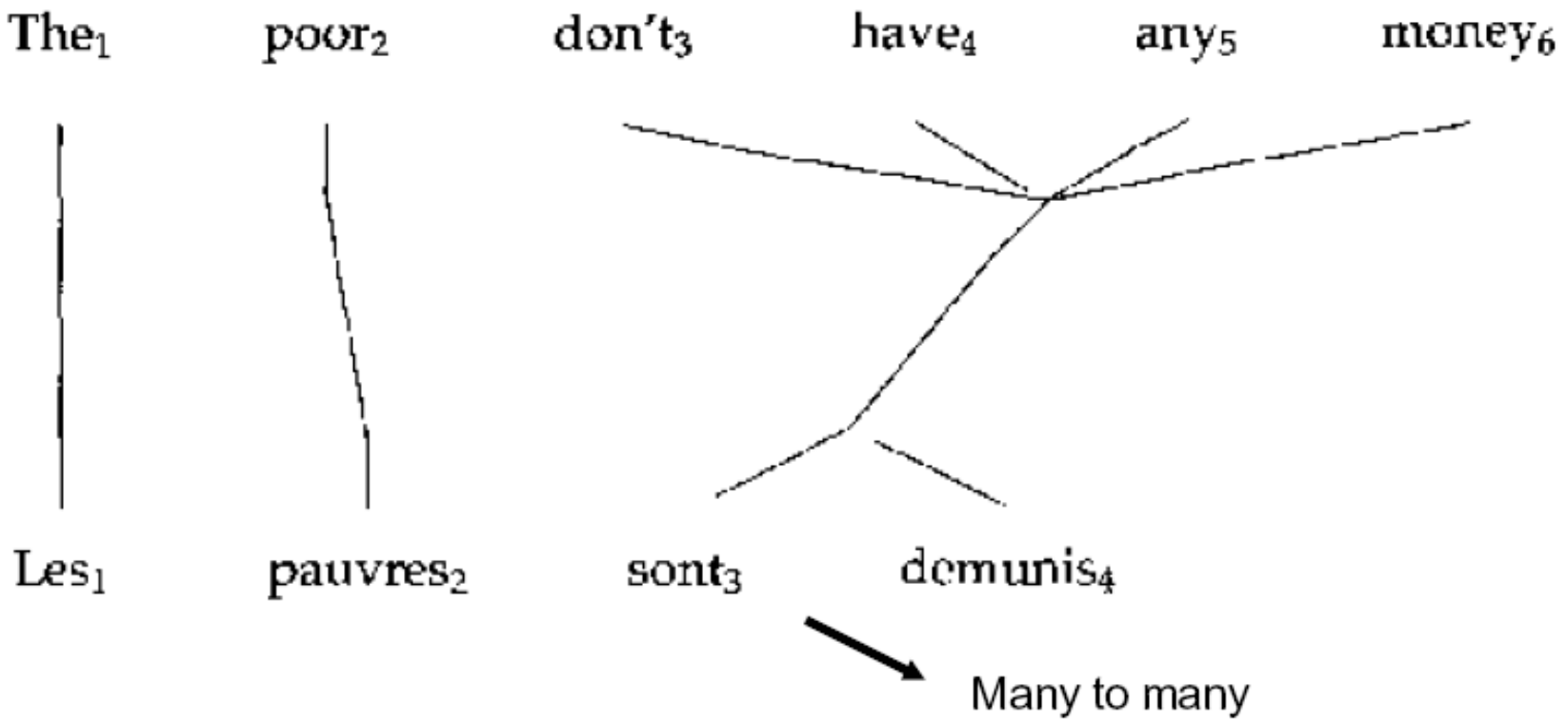
Word alignment - harder



Word alignment - harder



Word alignment - hard

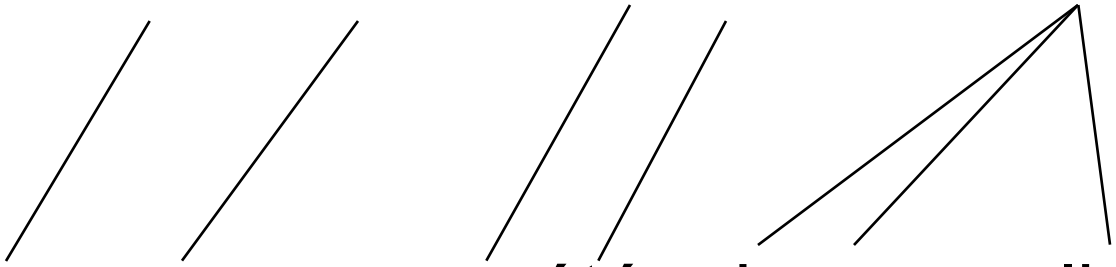


- A line group linking a minimal subset of words is called a 'cept' in the IBM work

Word alignment - encoding

0 1 2 3 4 5 6

- e_0 And the program has been implemented

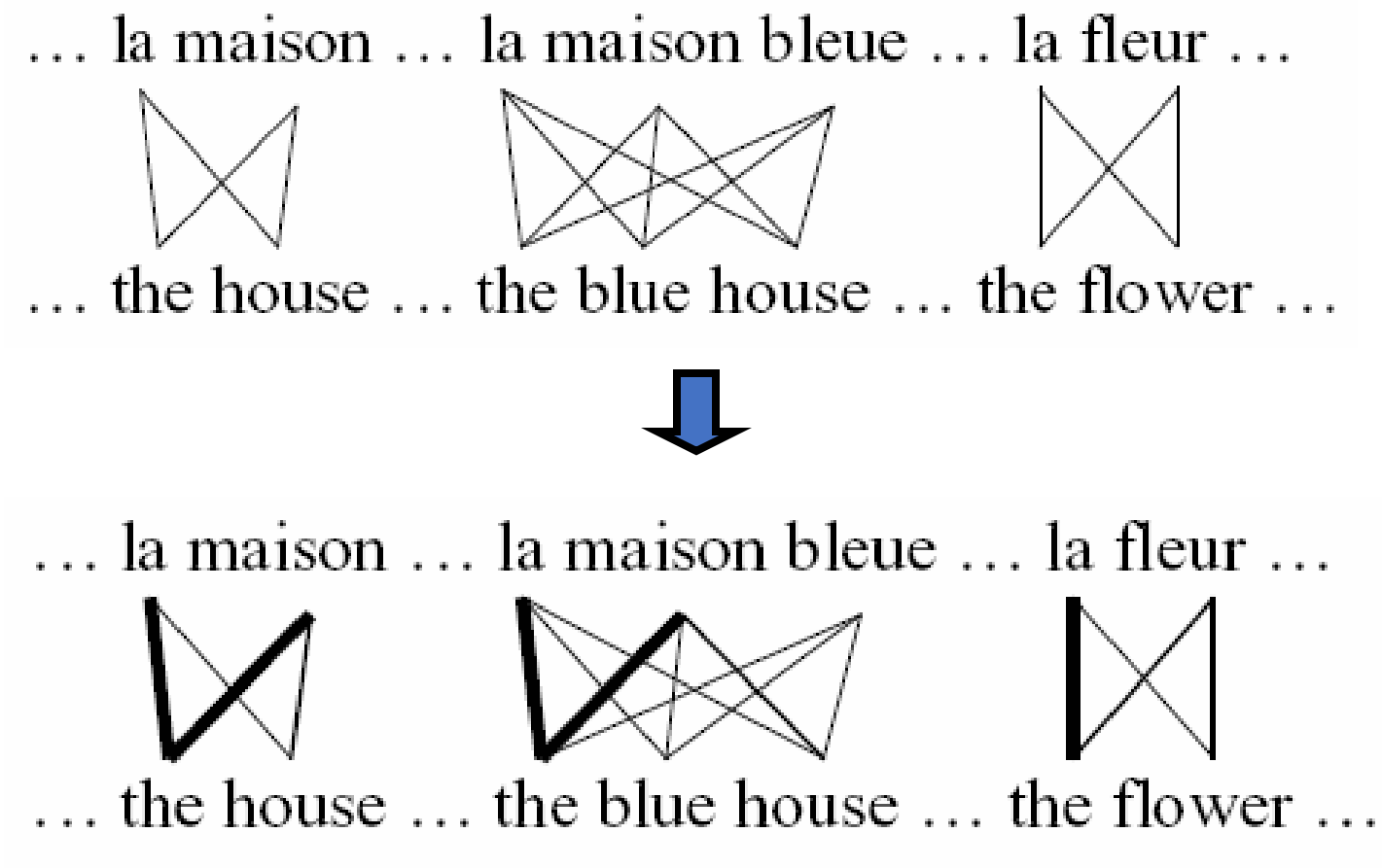


- f_0 Le programme a été mis en application

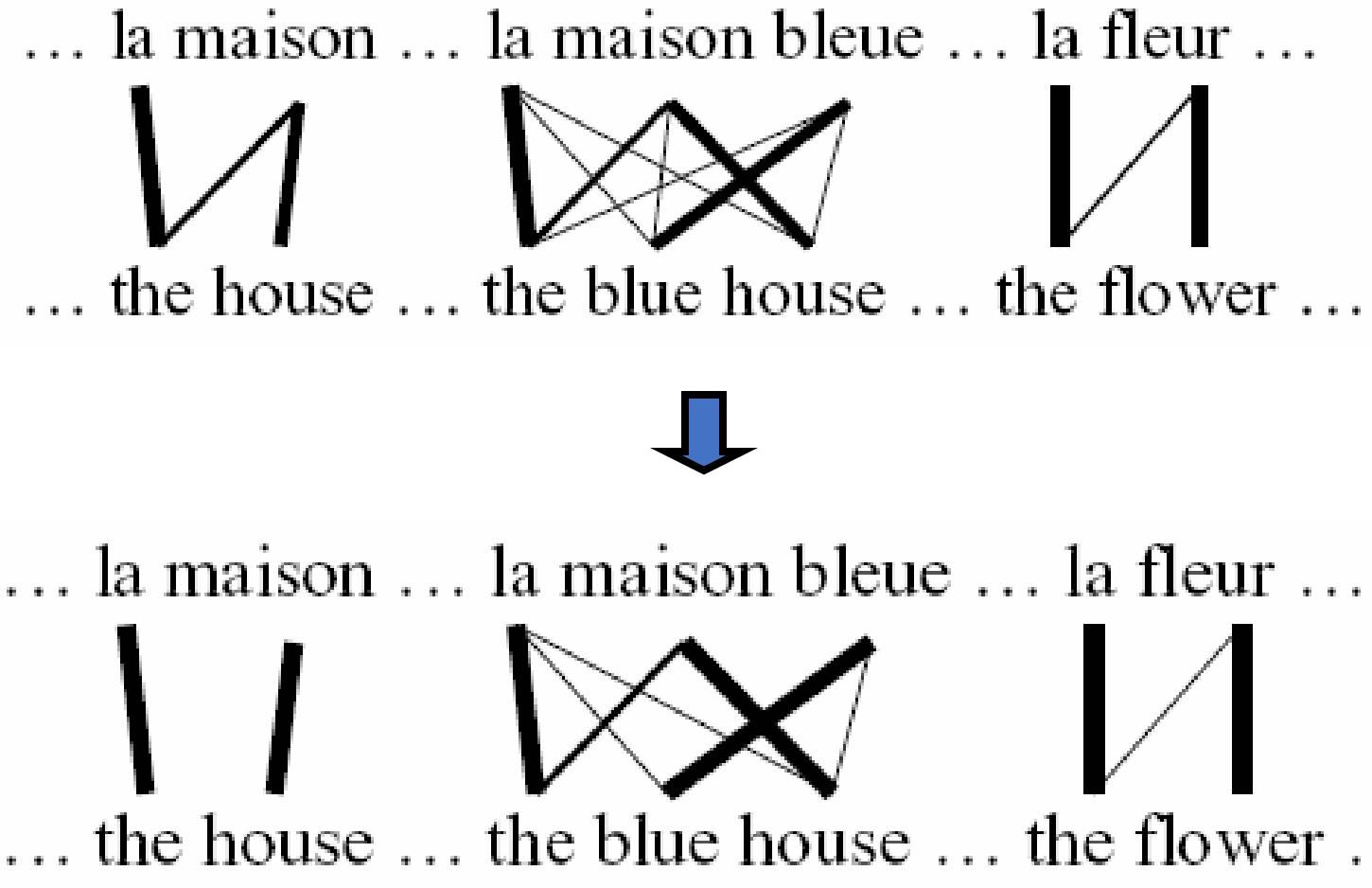
0 1 2 3 4 5 6 7

- Linear notation:
  - f_0(1) Le(2) programme(3) a(4) été(5) mis(6) en(6) application(6)
  - e_0 And(0) the(1) program(2) has(3) been(4) implemented(5,6,7)

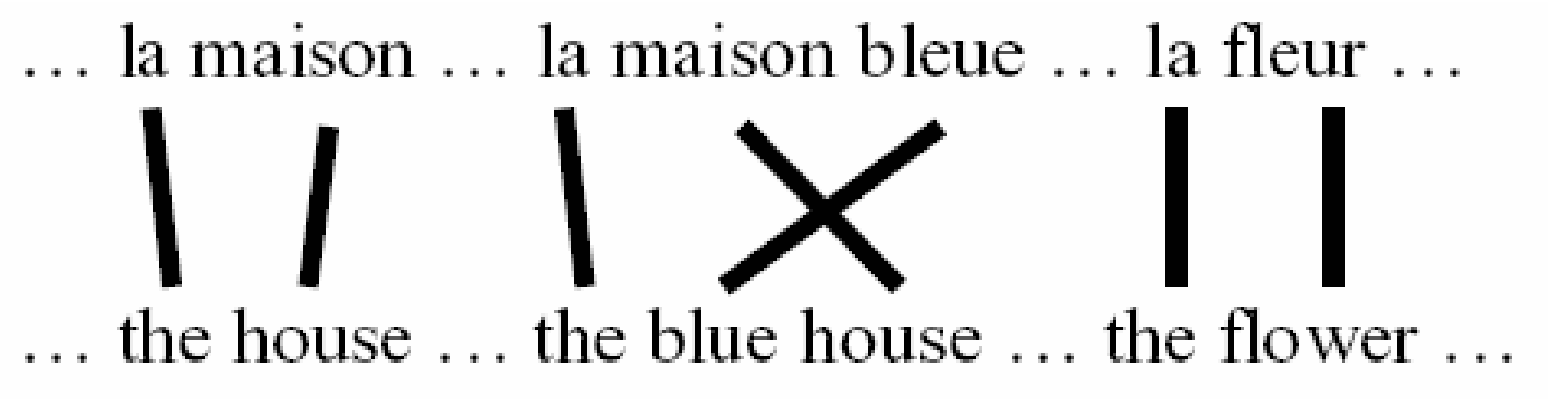
Word alignment learning with EM



Word alignment learning with EM



Word alignment learning with EM



EM algorithm

- Start with sentence-aligned corpus.
- Let (E,F) be a pair of sentences (actually, a bead).

1. Initialize p(f|e) randomly, $f \in F$, $e \in E$.
2. Compute expected counts over the corpus:

$$c(f,e) = \Sigma_{(E,F);e \in E,f \in F}\ p(f|e)$$

$\forall$ aligned pair (E,F), find if e in E and f in F; if yes, add p(f|e).

3. Re-estimate:

$p(f|e) = c(f,e) / c(e)$ $\quad [c(e) = \Sigma_f\ c(f,e)]$

4. Iterate until change of p(f|e) is small.

Best Alignment

Select, for each (E,F),

$$A = \text{argmax}_A P(A|F,E) = \text{argmax}_A P(F,A|E)/P(F) =$$

$$\text{argmax}_A P(F,A|E) = \text{argmax}_A (\varepsilon / (I+1)^m \prod_{j=1..m} p(f_j|e_{a_j}))$$

$$= \text{argmax}_A \prod_{j=1..m} p(f_j|e_{a_j})$$

- Use dynamic programming, Viterbi-like algorithm.
- Recompute p(f|e)

Elements of a Translation Model

- Assuming that
  - Individual translations are independance
  - 1 English word - n French words
  - 1 French word - (0-1) English word

$$P(f \mid e) = \frac{1}{Z} \sum_{a_1}^{l} \cdots \sum_{a_{m=0}}^{l} \prod_{j=1}^{m} P(f_j \mid e_{a_j})$$

- f_j - word j in f;
- a_j - position in e that f_j is aligned with
- e_{aj} - word in e that f_j is aligned with
- Z is a normalization constant
- a_j = 0: word j in the French sentence is aligned with the empty *cept* (no overt translation)
- m – length of f

Exercise

- Given the following bilingual dataset:
  - Mike yêu Jane. Mike loves Jane.
  - Jane yêu hoa. Jane loves flowers.
  - Jane thích đọc sách. Jane likes reading.
- Compute the following translation probabilities:
  - P(“Mike thích đọc sách| Mike likes reading”)
  - P(“Mike yêu hoa| Mike likes flowers”)

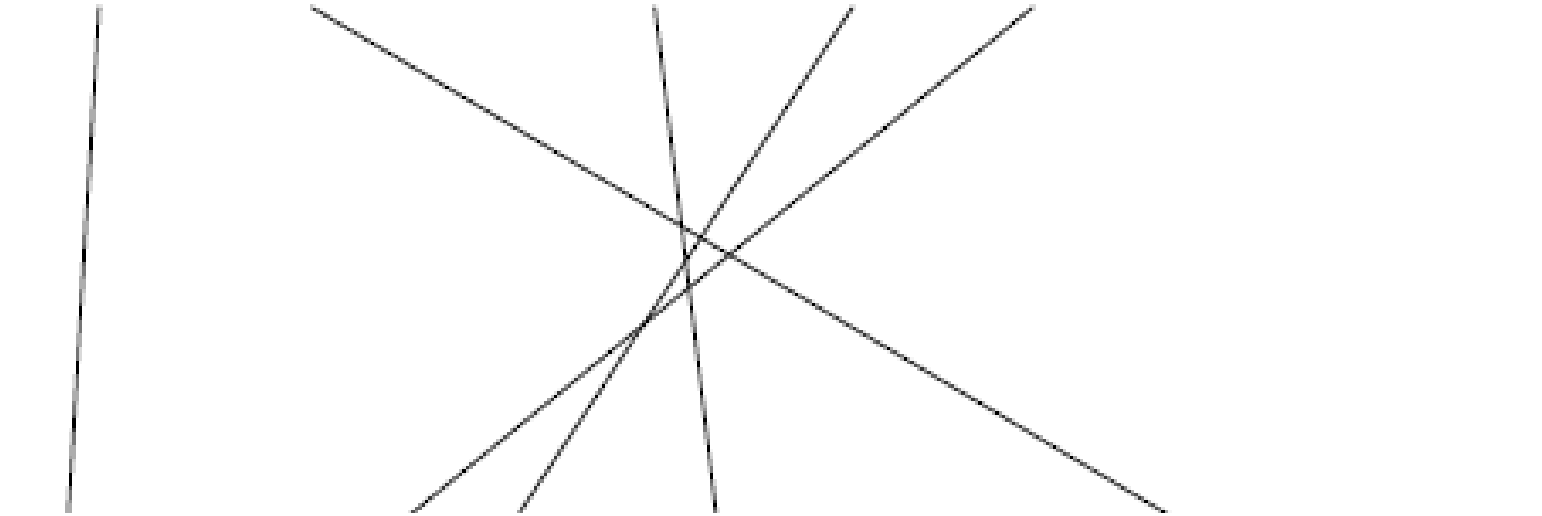
Machine translation using the syntax

Why Syntax?

- Need much more grammatical output
- Need accurate control over re-ordering
- Need accurate insertion of function words
- Word translations need to depend on grammatically-related word

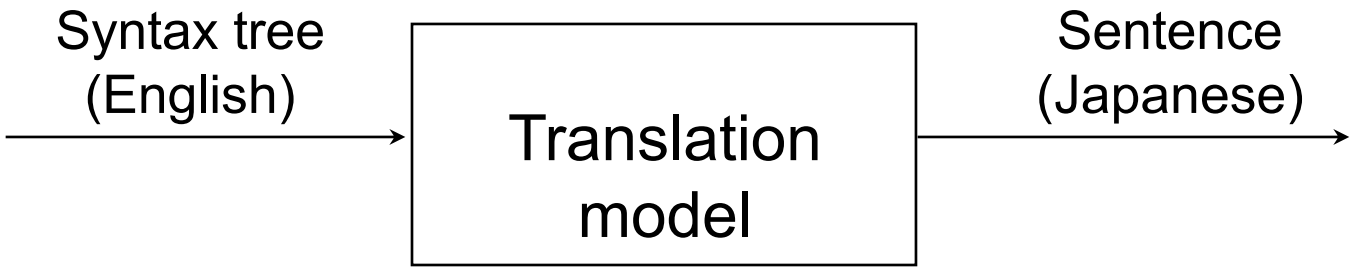
Yamada and Knight (2001): The need for phrasal syntax

- He adores listening to music.

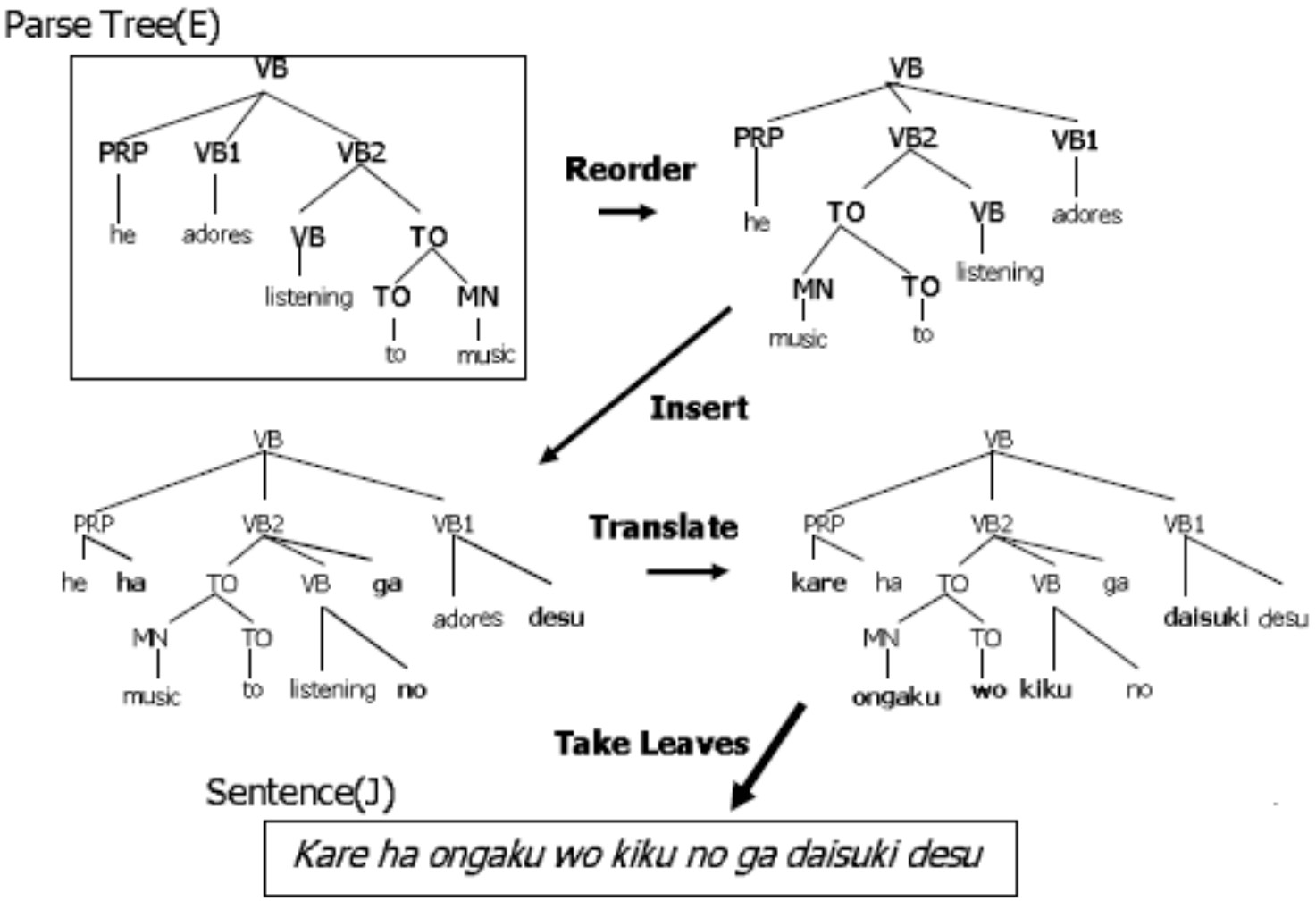


Syntax-based model

- Preprocess English by a parser
- Probabilistic operations on a parse-tree
  1. Reorder child nodes
  2. Insert extra nodes
  3. Translate leaf words

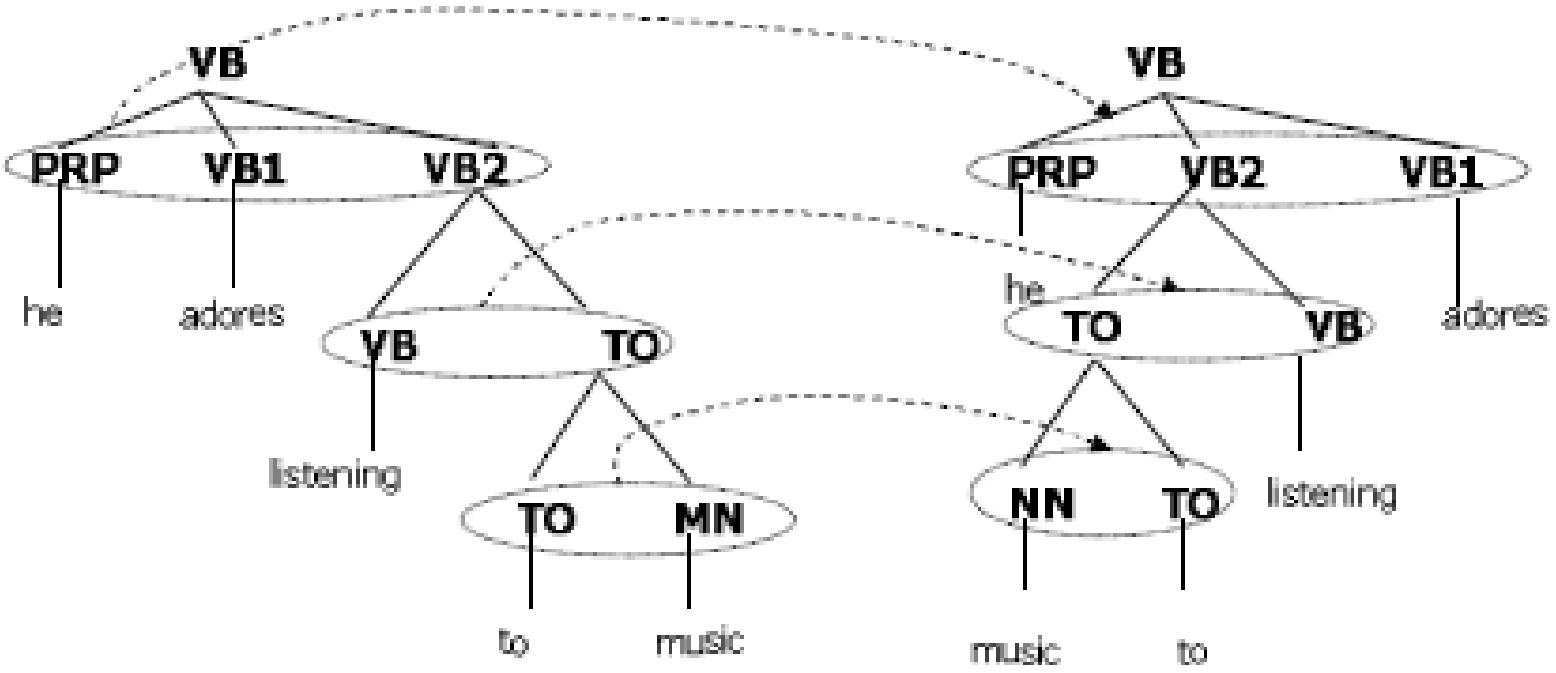


Parse Tree(E) → Sentence (J)



1. Reorder

Conditional feature = child label sequence



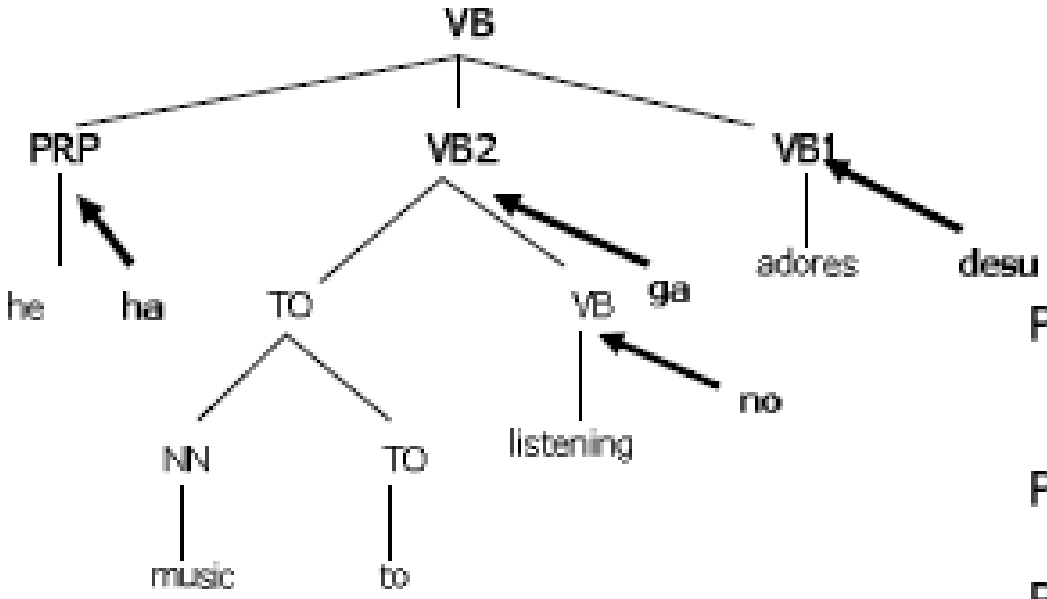
P(PRP VB1 VB2 — PRP VB2 VB1) = 0.723
P(VB TO — TO VB) = 0.749
P(TO NN — NN TO) = 0.893

Parameter Table: Reorder

Original order	Coming back	P(Reorder|Original order)
PRP VB1 VB2	PRP VB1 VB2	0.074
	PRP VB2 VB1	**0.723**
	VB1 PRP VB2	0.061
	VB1 VB2 PRP	0.037
	VB2 PRP VB1	0.083
	VB2 VB1 PRP	0.021
VB TO	VB TO	0.107
	TO VB	**0.893**
TO NN	TO NN	0.251
	NN TO	**0.749**

2. Insert

Conditional Feature = parent label & node label (position) & none (word selection)



P(none|TOP-VB) = 0.735

⋮

P(right|VB-PRP)* P(ha) = 0.652 * 0.219

P(right|VB-VB) * P (ga) = 0.252 * 0.062

⋮

P(none|TO-TO) = 0.900

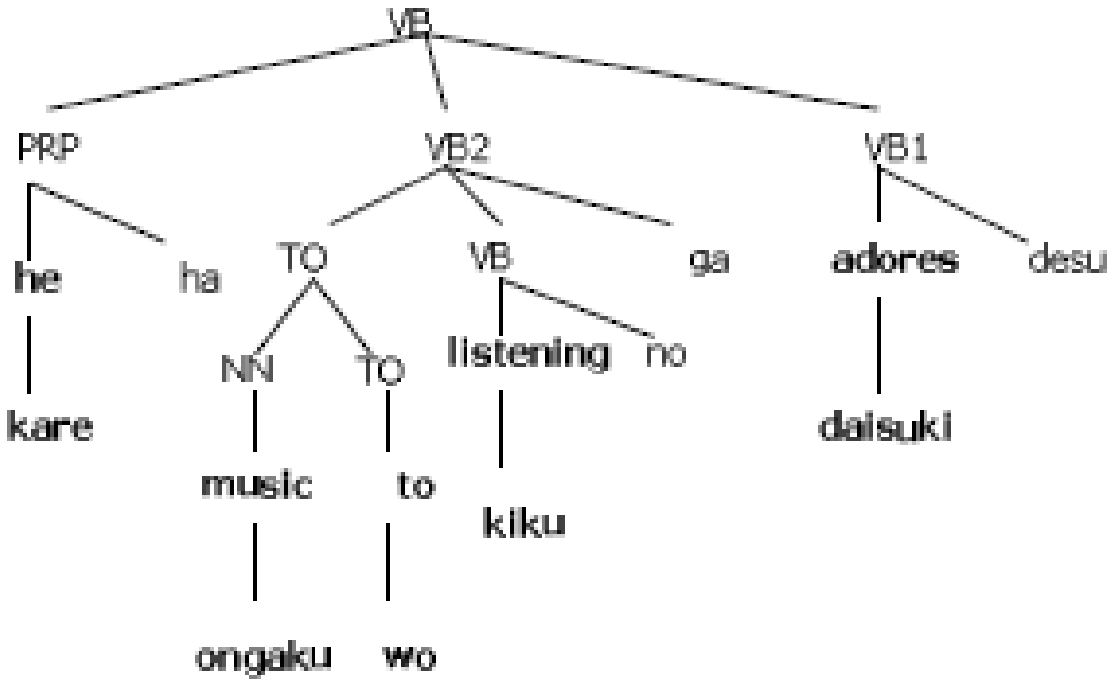
Parameter table: insert

Parent label node level	TOP VB	VB VB	VB TO	TO TO	TO NN	TO NN
P (none)	0.735	0.687	0.344	0.700	0.900	0.800
P (left)	0.004	0.061	0.004	0.030	0.003	0.096
P (right)	0.260	0.252	0.652	0.261	0.097	0.104

W	P (insert-w)
ha	0.219
ta	0.131
wo	0.099
no	0.094
ni	0.090
te	0.079
ga	0.062
|	|
desu	0.0007
|	|

3. Translate

Conditional feature = word identity (English)



P (he—— kare) = 0.952
P (music ——ongaku) =0.900
P (to— wo) = 0.038
P (listening —kiku) = 0.333
P (adore —daisuki) = 1.000

Parameter table: Translate

Note: Translation to NULL = deletion

E	adores	he	listening	music	to
J	daisuki 1.000	kare 0.952 NULL 0.016 nani 0.005 da 0.003 shi 0.003	kiku 0.333 kii 0.333 mi 0.333	ongaku 0.900 naru 0.100	ni 0.216 NULL 0.204 to 0.133 no 0.046 wo 0.038

Experiment

- Training Corpus: J-E 2K sentence pairs
- J: Tokenized by Chasen [Matsumoto, et al., 1999]
- E: Parsed by Collins Parser [Collins, 1999]
  - Trained: 40K Treebank, Accuracy: ~90%
- E: Flatten parse tree
  - To Capture word-order difference (SVO->SOV)
- EM Training: 20 Iterations
  - 50 min/iter (Sparc 200Mhz 1-CPU) or
  - 30 sec/iter (Pentium3 700Mhz 30-CPU)

Result

- Average by 3 humans for 50 sentences
- ok(1.0), not sure (0.5), wrong (0.0)
- Precision only

	Average	#perf sent
Y/K models	0.582	10
IBM model 5	0.431	0

Neural Machine Translation

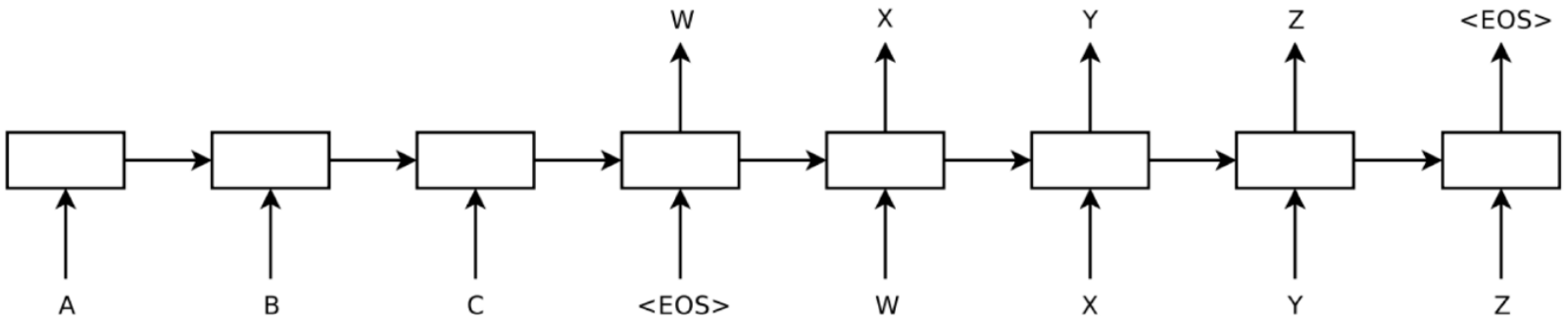
Neural Machine Translation

- Neural Machine Translation (NMT) is a way to do Machine Translation with a *single neural network*

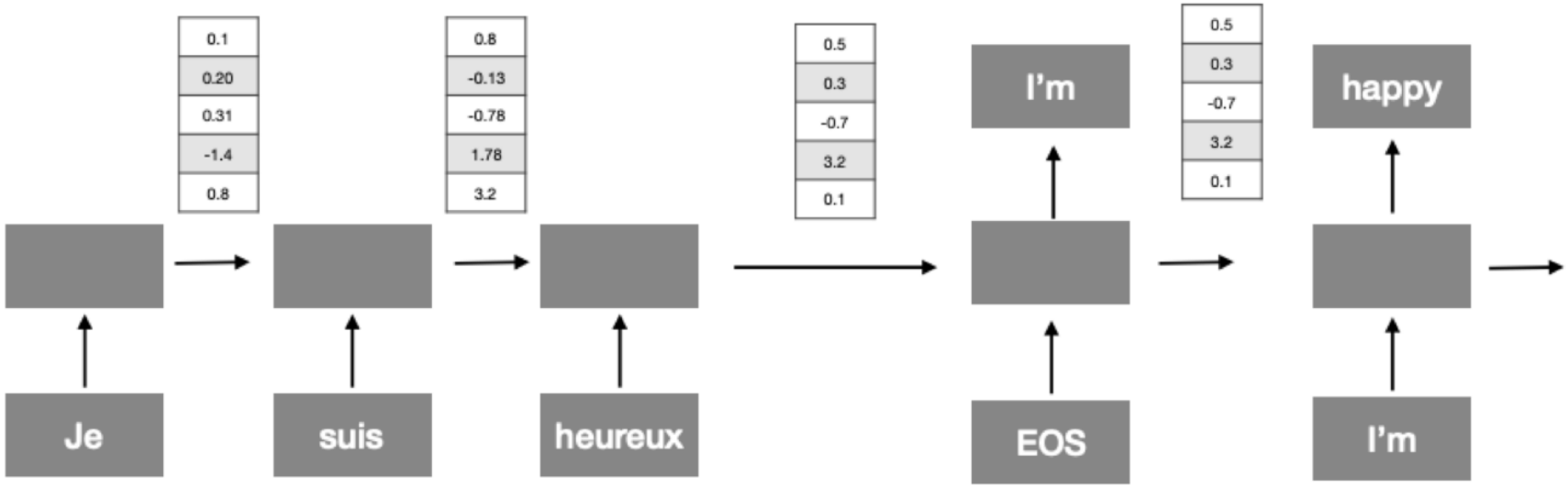
- The neural network architecture is called sequence-to-sequence (seq2seq) and it involves *two* RNNs.

Encoder-decoder Framework

"Sequence to Sequence Learning with Neural Networks", 2014

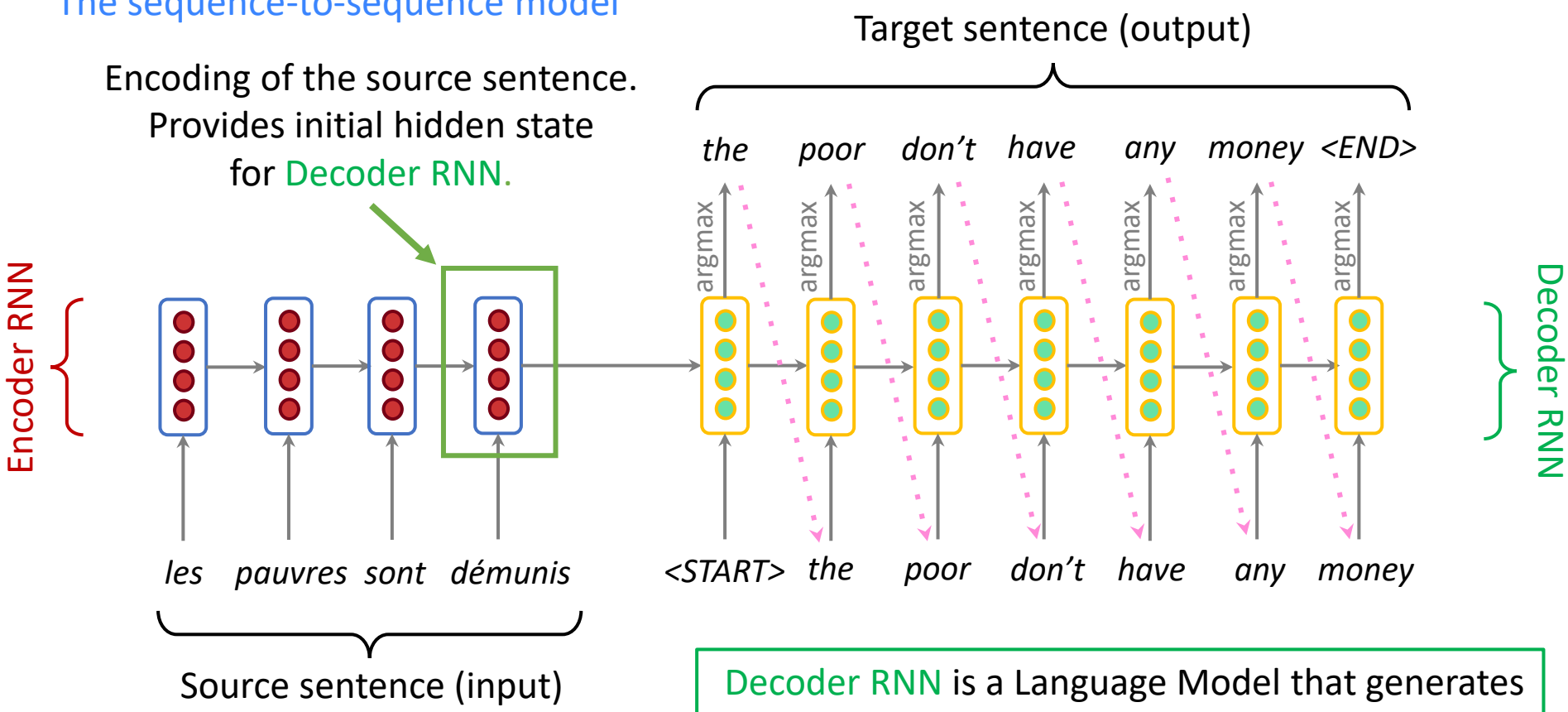


Encoder-decoder Framework



Neural Machine Translation (NMT)

The sequence-to-sequence model



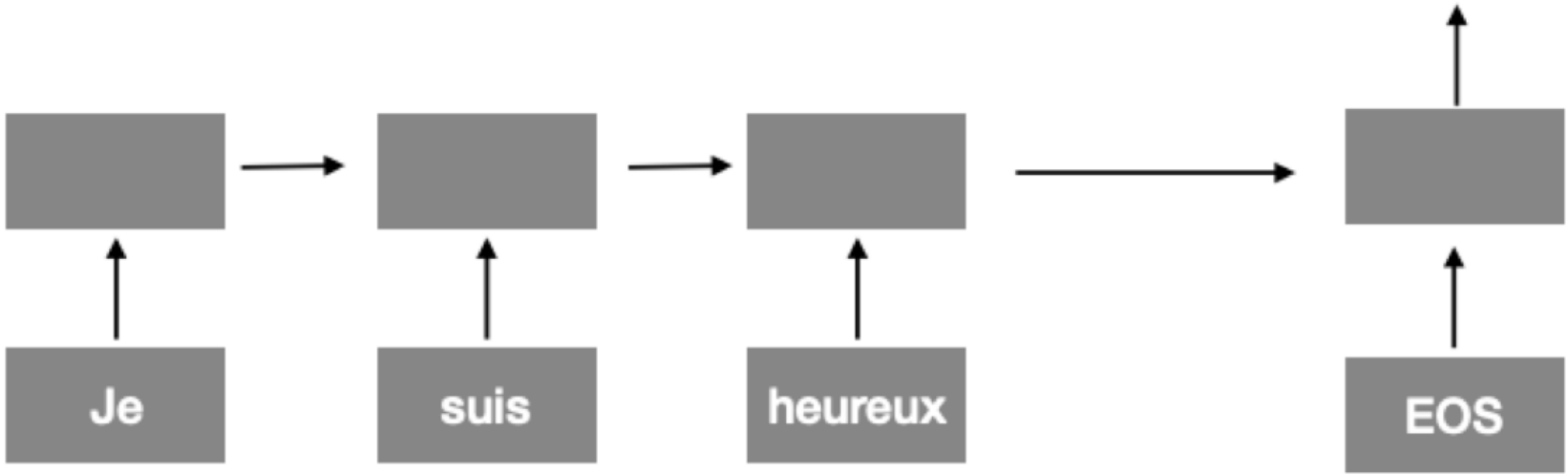
Encoder RNN produces an encoding of the source sentence.

Decoder RNN is a Language Model that generates target sentence conditioned on encoding.

Note: This diagram shows **test time** behavior: decoder output is fed in ····→ as next step's input

Training

- As with other RNN models, we can train by minimizing the loss function between what we predict at each step and its correct value



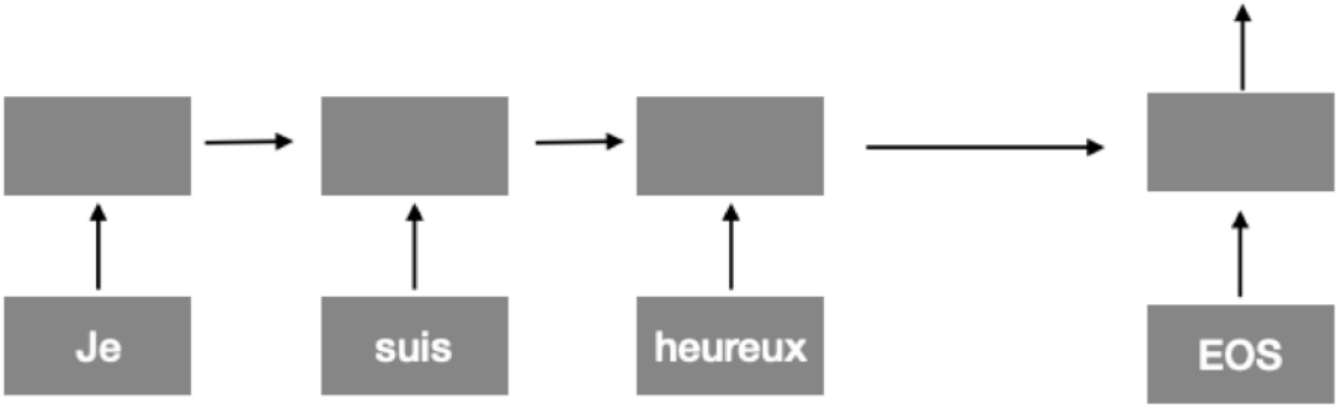
Training

Truth

I'm	you	are	the	...
1	0	0	0	0

Predicted

I'm	you	are	the	...
0.03	0.05	0.02	0.01	0.009

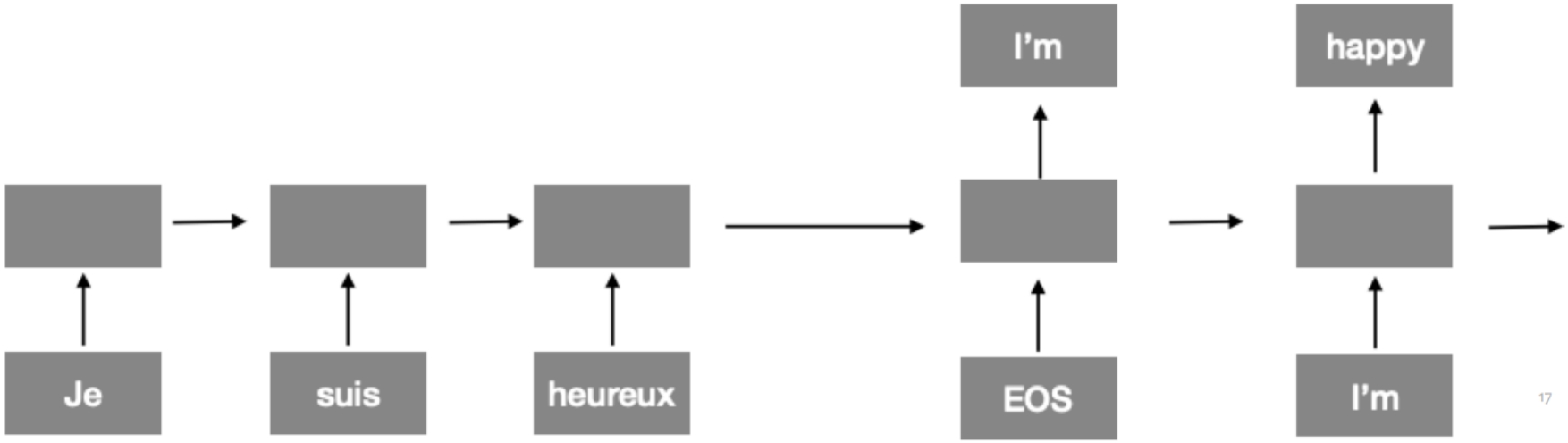


Truth

happy	great	bad	ok	…
1	0	0	0	0

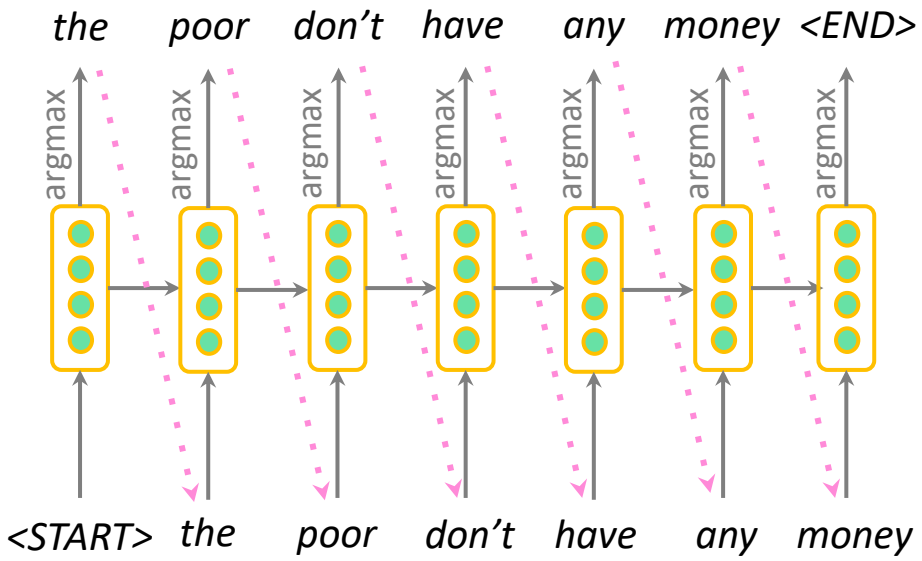
Predicted

happy	great	bad	ok	…
0.13	0.08	0.01	0.03	0.009



Better-than-greedy decoding?

- We showed how to generate (or "decode") the target sentence by taking argmax on each step of the decoder



- This is greedy decoding (take most probable word on each step)
- **Problems?**

Better-than-greedy decoding?

- Greedy decoding has no way to undo decisions!
  - *les pauvres sont démunis (the poor don't have any money)*
  - → *the* ____
  - → *the poor* ____
  - → *the poor are* ____
- Better option: use beam search (a search algorithm) to explore *several* hypotheses and select the best one

Decoding based on Beam Search

- Ideally we want to find *y* that maximizes

$$P(y|x) = P(y_1|x)\, P(y_2|y_1, x)\, P(y_3|y_1, y_2, x) \ldots, P(y_T|y_1, \ldots, y_{T-1}, x)$$

- We could try enumerating all *y* → too expensive!
  - Complexity $O(V^T)$ where *V* is vocab size and *T* is target sequence length

- **Beam search**: On each step of decoder, keep track of the *k* most probable partial translations
  - *k* is the beam size (in practice around 5 to 10)
  - Not guaranteed to find optimal solution
  - But much more efficient!

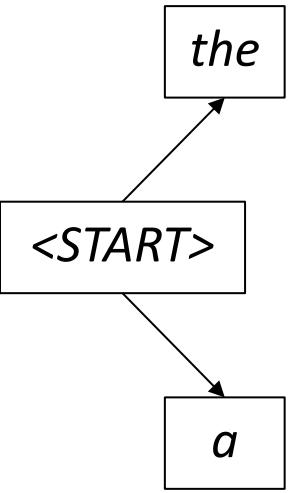
Decoding based on Beam Search: Example

Beam size = 2

<START>

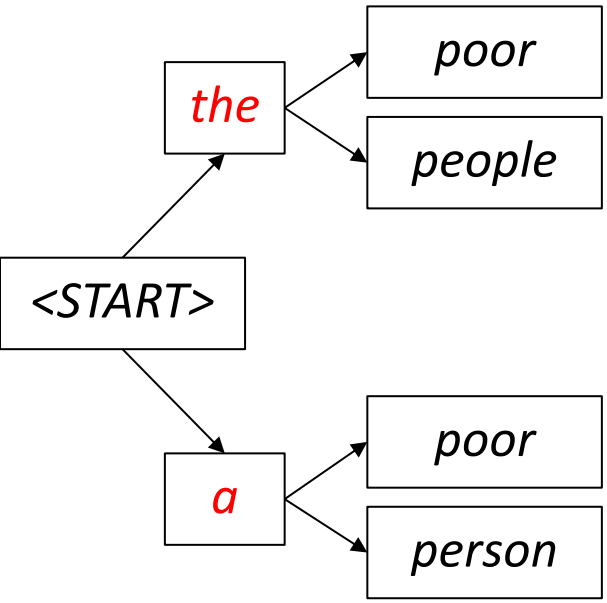
Decoding based on Beam Search: Example

Beam size = 2



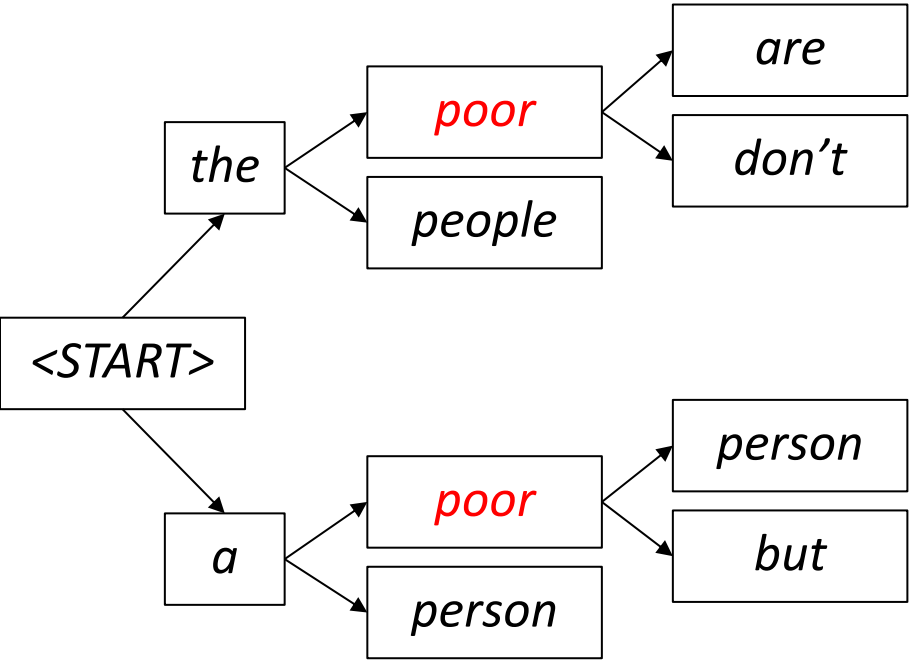
Decoding based on Beam Search: Example

Beam size = 2



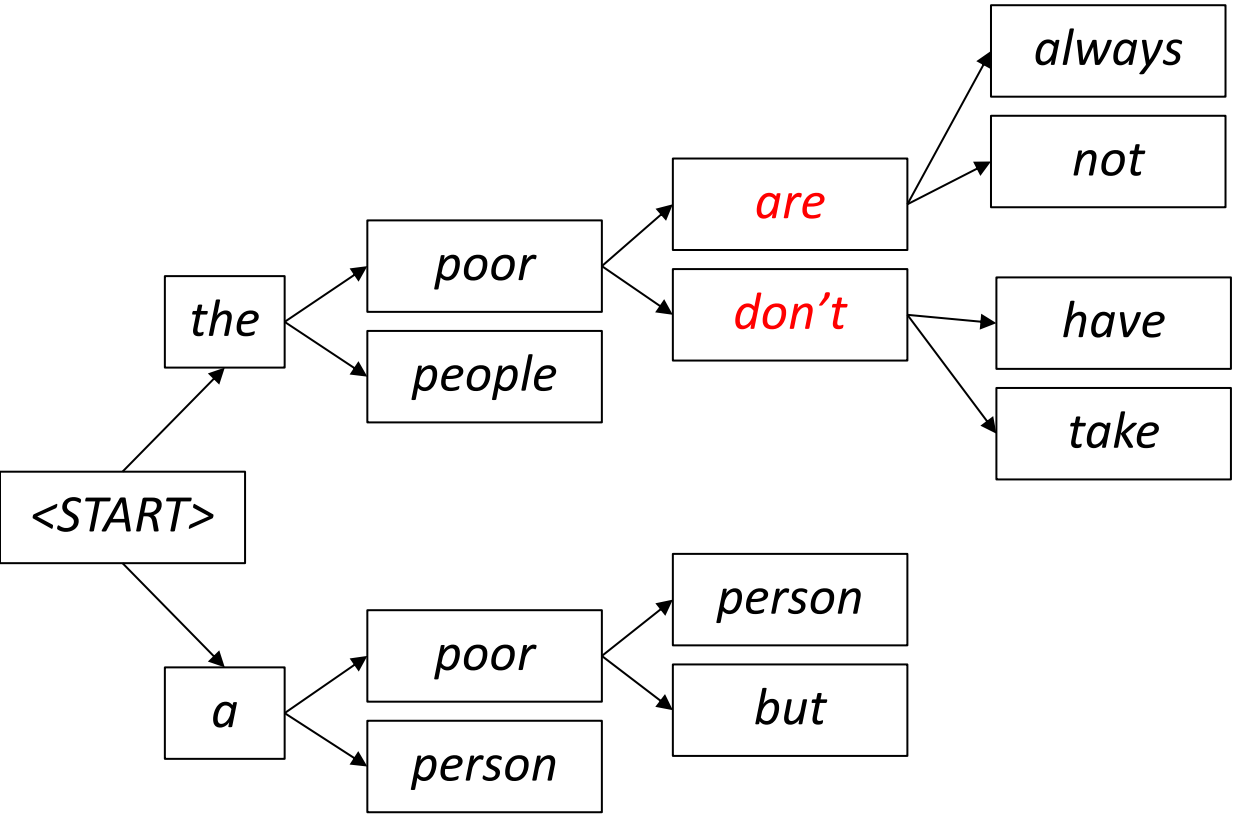
Decoding based on Beam Search: Example

Beam size = 2



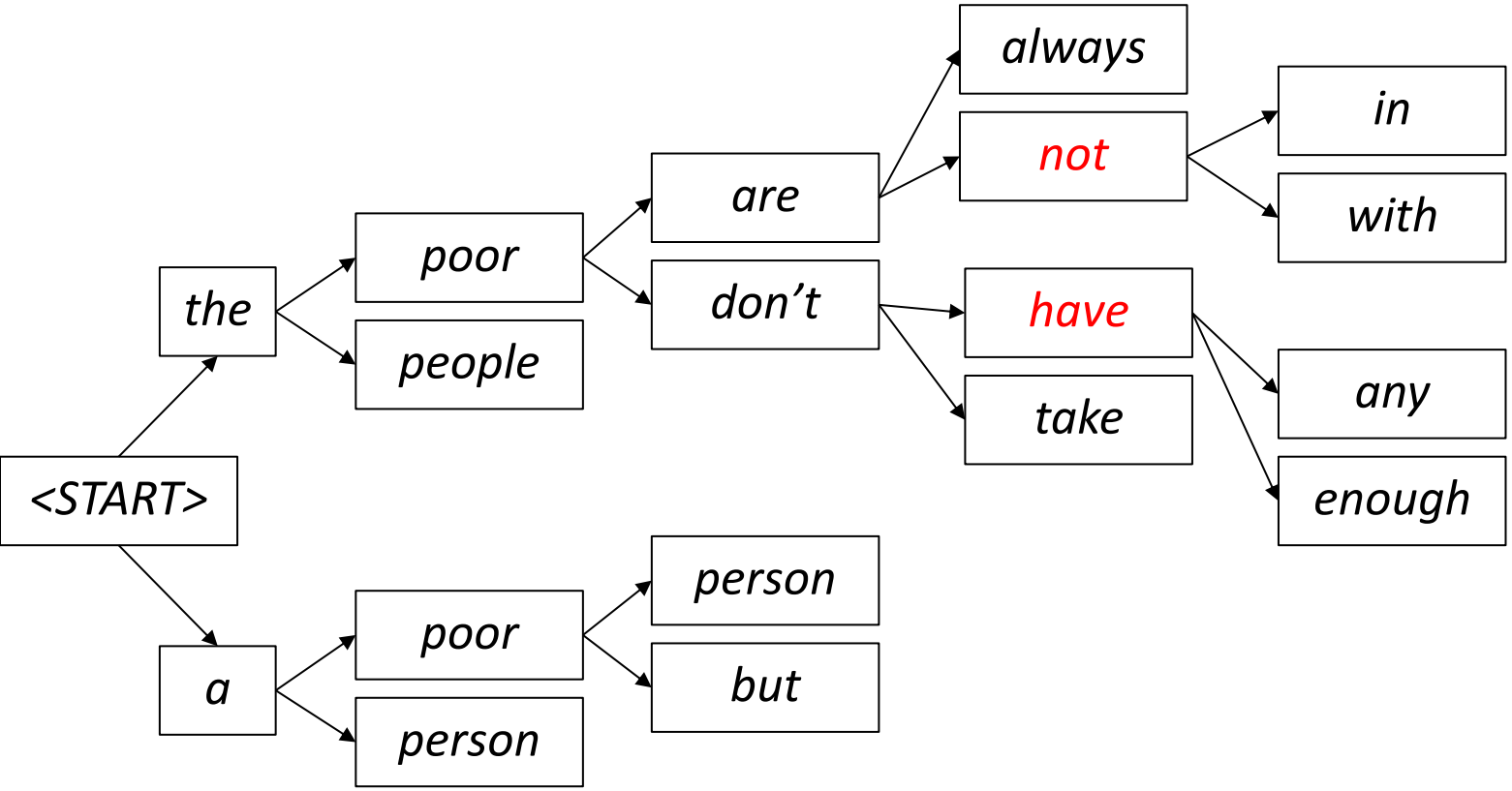
Decoding based on Beam Search: Example

Beam size = 2



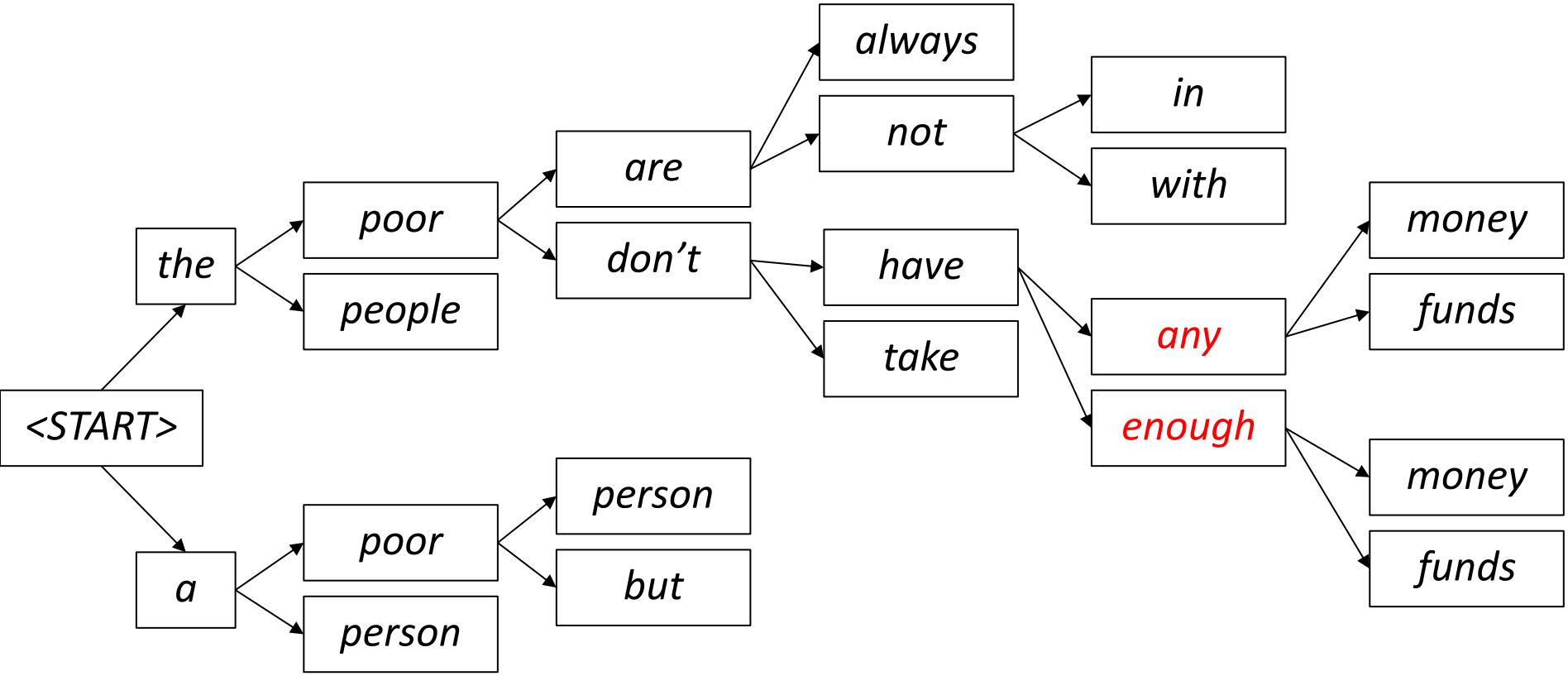
Decoding based on Beam Search: Example

Beam size = 2



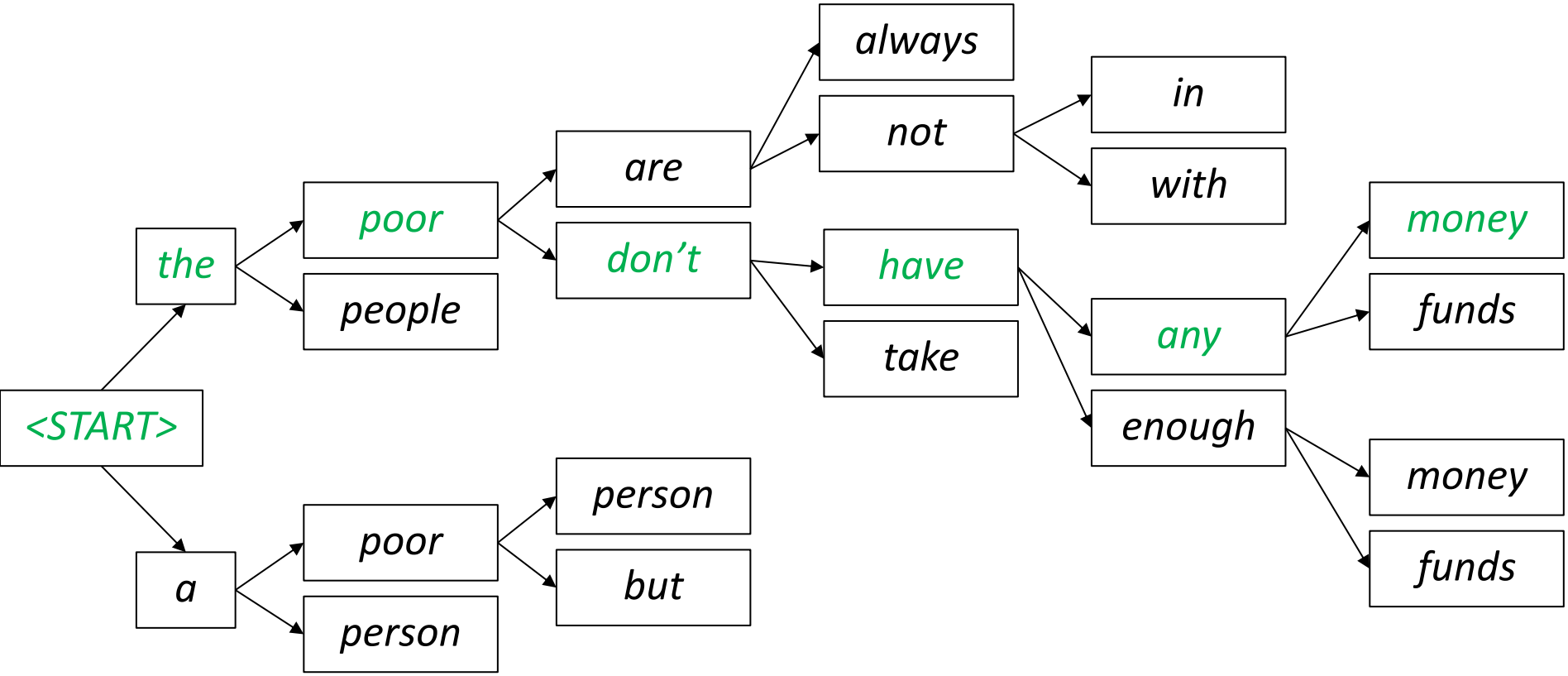
Decoding based on Beam Search: Example

Beam size = 2



Decoding based on Beam Search: Example

Beam size = 2



Beam search: Stopping criteria

- In greedy decoding, we typically decode until the model generates the <END> token
- Example : *<START> he hit me with a pie <END>*
- In beam search decoding, different hypotheses may generate the <END> token at different time steps.
  - When a hypothesis generates <END>, it is considered completed.
  - Set it aside and continue exploring other hypotheses through beam search.
- Typically, we continue beam search until either:
  - A time step T is reached (where T is a predefined threshold), or
  - At least n hypotheses have been completed (where n is a predefined threshold).

Beam search: Final step

- We have our list of completed hypotheses.
- How to select top one with highest score?

- Each hypothesis $y_1, \ldots, y_t$ on our list has a score

$$\text{score}(y_1, \ldots, y_t) = \log P_{\text{LM}}(y_1, \ldots, y_t | x) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i | y_1, \ldots, y_{i-1}, x)$$

- Problem with this: longer hypotheses have lower scores

- Fix: Normalize by length. Use this to select top one instead:

$$\frac{1}{t} \sum_{i=1}^{t} \log P_{\text{LM}}(y_i | y_1, \ldots, y_{i-1}, x)$$

Advantages of NMT

Compared to SMT, NMT has many advantages:

- Better performance
  - More fluent
  - Better use of context
  - Better use of phrase similarities
- A single neural network to be optimized end-to-end
  - No subcomponents to be individually optimized
- Requires much less human engineering effort
  - No feature engineering
  - Same method for all language pairs

Disadvantages of NMT

Compared to SMT:

- NMT is less interpretable
  - Hard to debug
- NMT is difficult to control
  - For example, can't easily specify rules or guidelines for translation
  - Safety concerns!

How do we evaluate Machine Translation?

BLEU (**Bil**ingual **E**valuation **U**nderstudy)

- BLEU compares the machine-written translation to one or several human-written translation(s), and computes a similarity score based on:
  - *n*-gram precision (usually up to 3 or 4-grams)
  - Penalty for too-short system translations

- BLEU is useful but imperfect
  - There are many valid ways to translate a sentence
  - So a good translation can get a poor BLEU score because it has low *n*-gram overlap with the human translation ☹

How do we evaluate Machine Translation?

- Precision n-gram = Number of correct predicted n-grams / Number of total predicted n-grams

- **Correct sentence:** *The guard arrived late because it was raining*
- **Predicted sentence:** *The guard arrived late because of the rain*

Precision 1-gram ($p1$) = 5 / 8
Precision 2-gram ($p2$)= 4 / 7
recision 3-gram ($p3$) = 3 / 6
Precision 4-gram ($p4$) = 2 / 5

- **Correct sentence** : Camping in Mù Cang Chải to admire the ripe rice season.
- **Predicted sentence** : Camping in Mù Cang Chải to enjoy the rice harvest season.

How do we evaluate Machine Translation?

- **Geometric Average Precision Scores**

$$
\begin{aligned}
\textit{Geometric Average Precision } (N) &= exp(\sum_{n=1}^{N} w_n \; log \; p_n) \\
&= \prod_{n=1}^{N} {p_n}^{w_n} \\
&= (p_1)^{\frac{1}{4}} \cdot (p_2)^{\frac{1}{4}} \cdot (p_3)^{\frac{1}{4}} \cdot (p_4)^{\frac{1}{4}}
\end{aligned}
$$

How do we evaluate Machine Translation?

- **Correct sentence:** *The guard arrived late because it was raining*
- **Predicted sentence:** *The guard*

  Precision 1-gram ($p1$) = 2 / 2

  Precision 2-gram ($p2$)= 1 / 1

  - Encourage the model to generate shorter outputs and achieve higher scores..
- **Brevity Penalty:** Punish overly short sentences

$$Brevity\ Penalty = \begin{cases} 1, & \text{if } c > r \\ e^{(1-r/c)}, & \text{if } c <= r \end{cases}$$

- c is *predicted length* = the number of words in the predicted sentence.
- r is *target length* = the number of words in the target sentence.

How do we evaluate Machine Translation?

- **BLEU score**

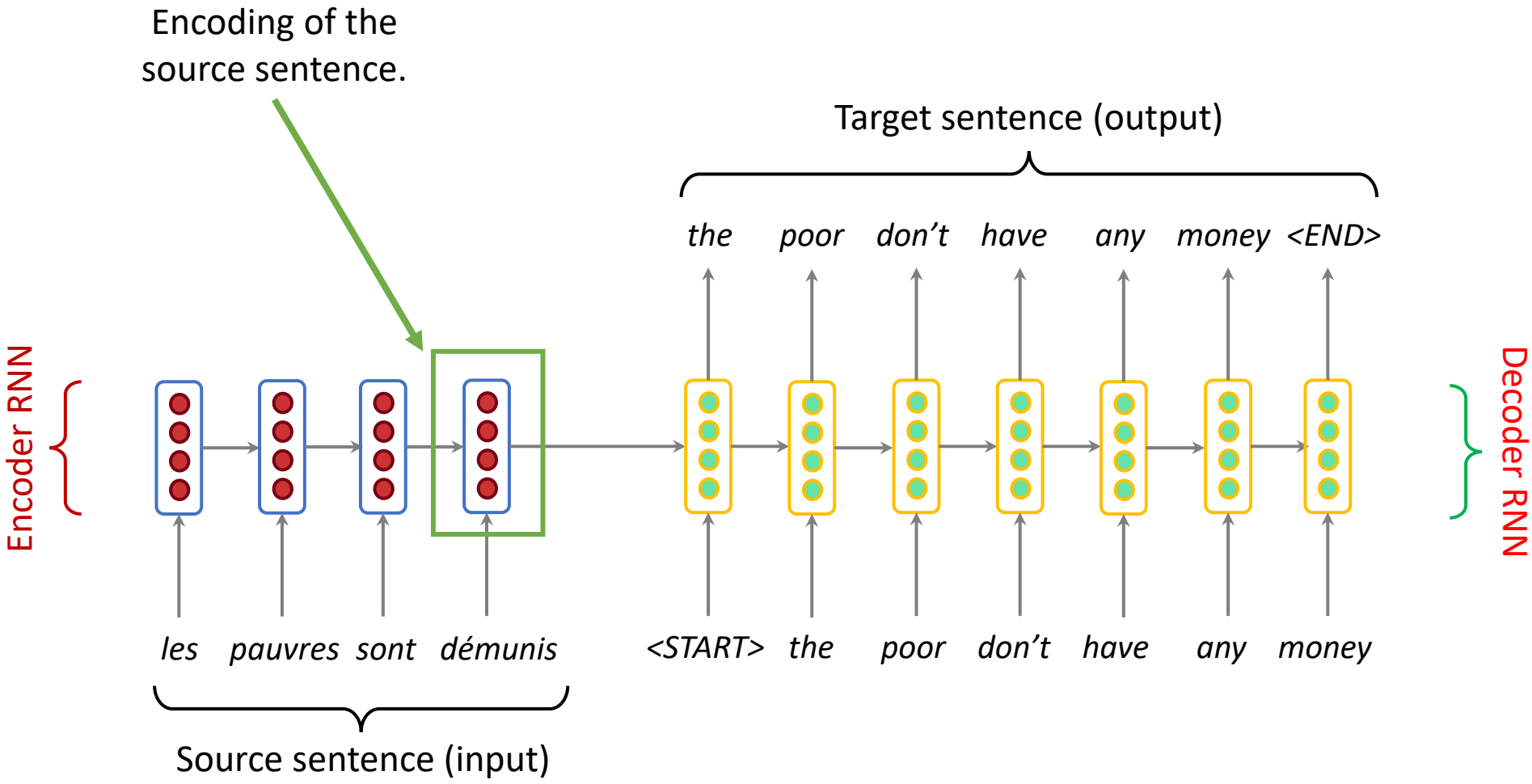
$$Bleu\ (N) = Brevity\ Penalty \cdot Geometric\ Average\ Precision\ Scores\ (N)$$

with N = number of gram

- Another BLEU formula

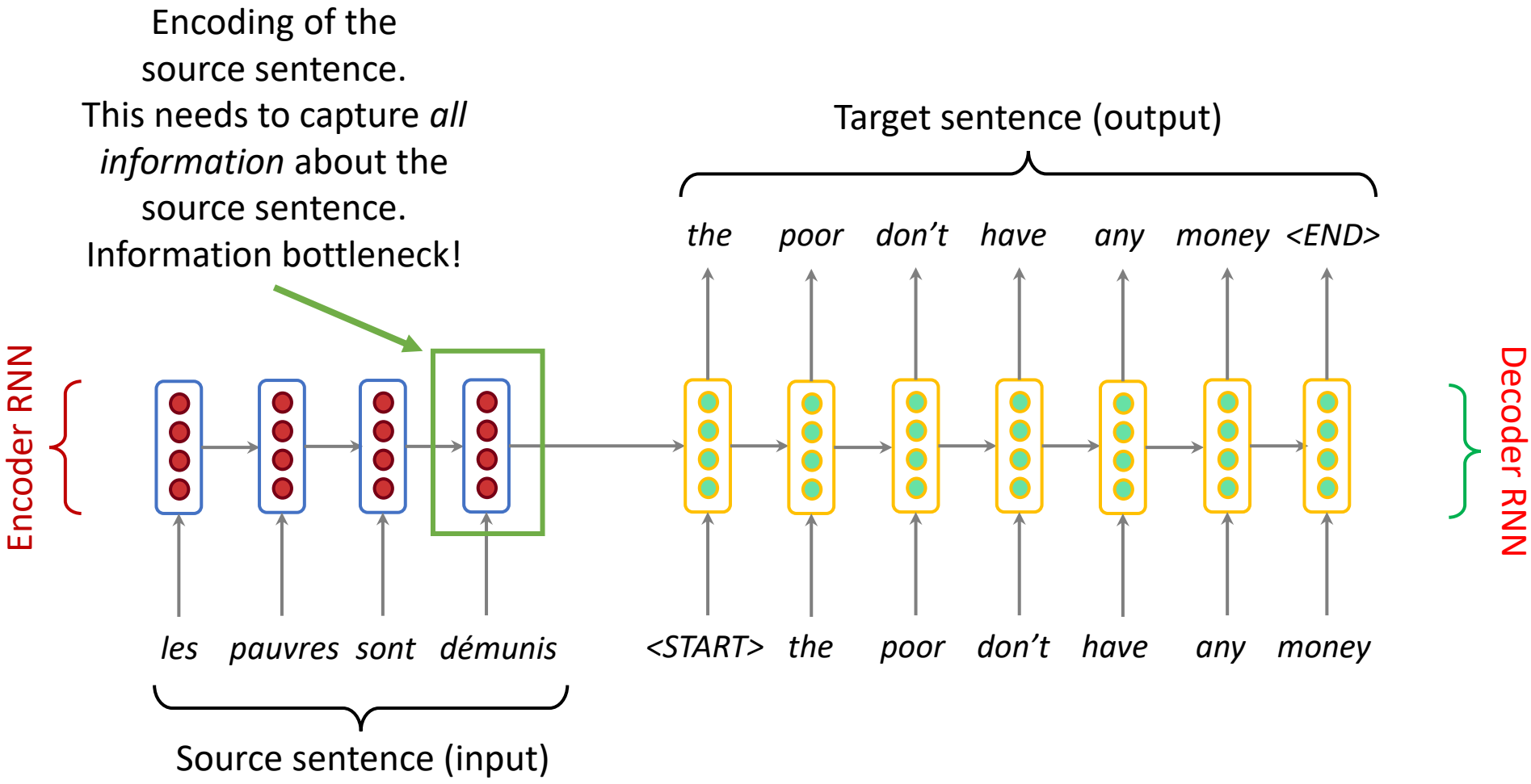
$$\begin{aligned} log\ Bleu &= min(1 - \frac{r}{c}, 0) + \sum_{n=1}^{4} \frac{log\ p_n}{4} \\ &= min(1 - \frac{r}{c}, 0) + \frac{log\ p_1 + log\ p_2 + log\ p_3 + log\ p_4}{4} \end{aligned}$$

Sequence-to-sequence: the bottleneck problem



Problems with this architecture?

Sequence-to-sequence: the bottleneck problem

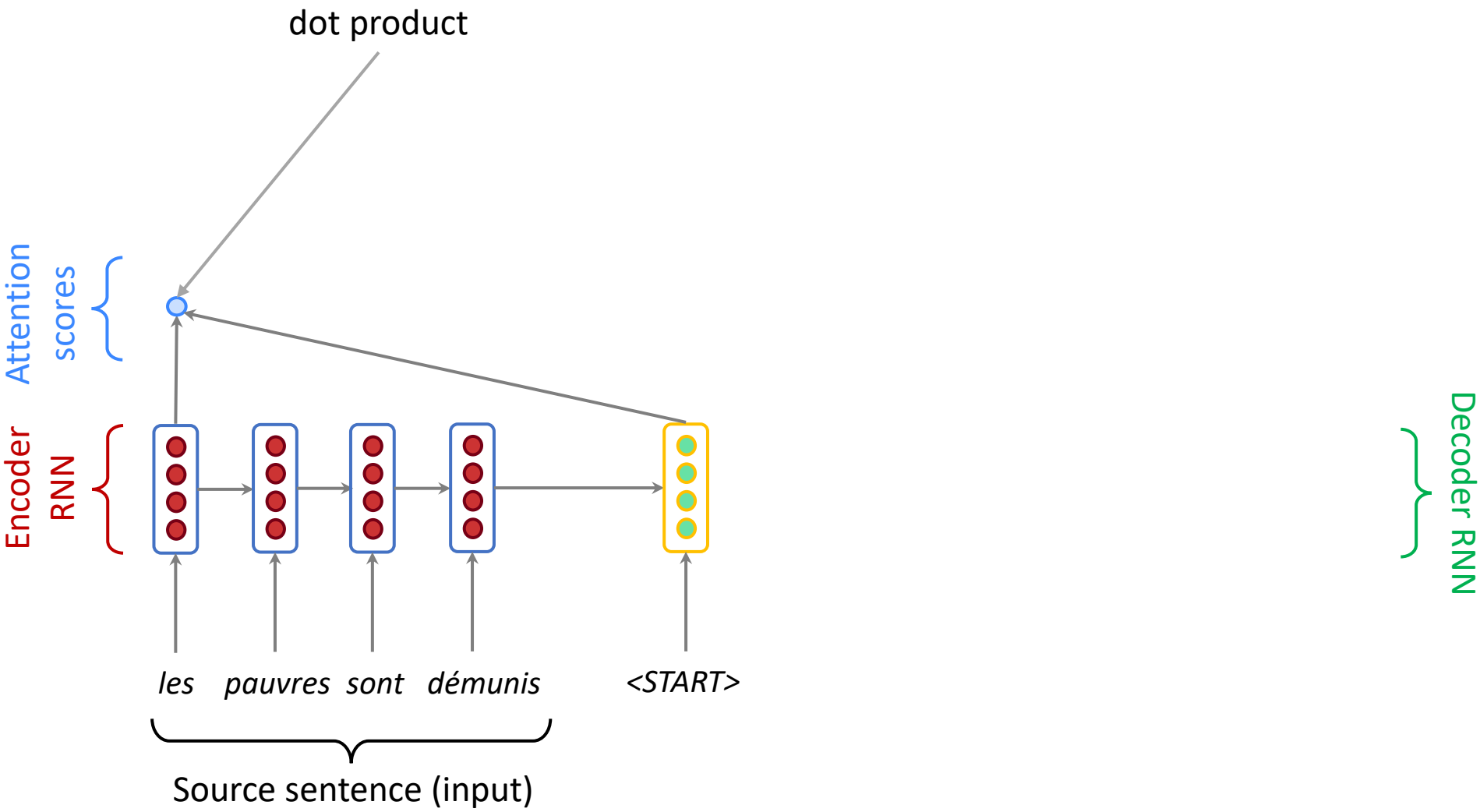


Attention

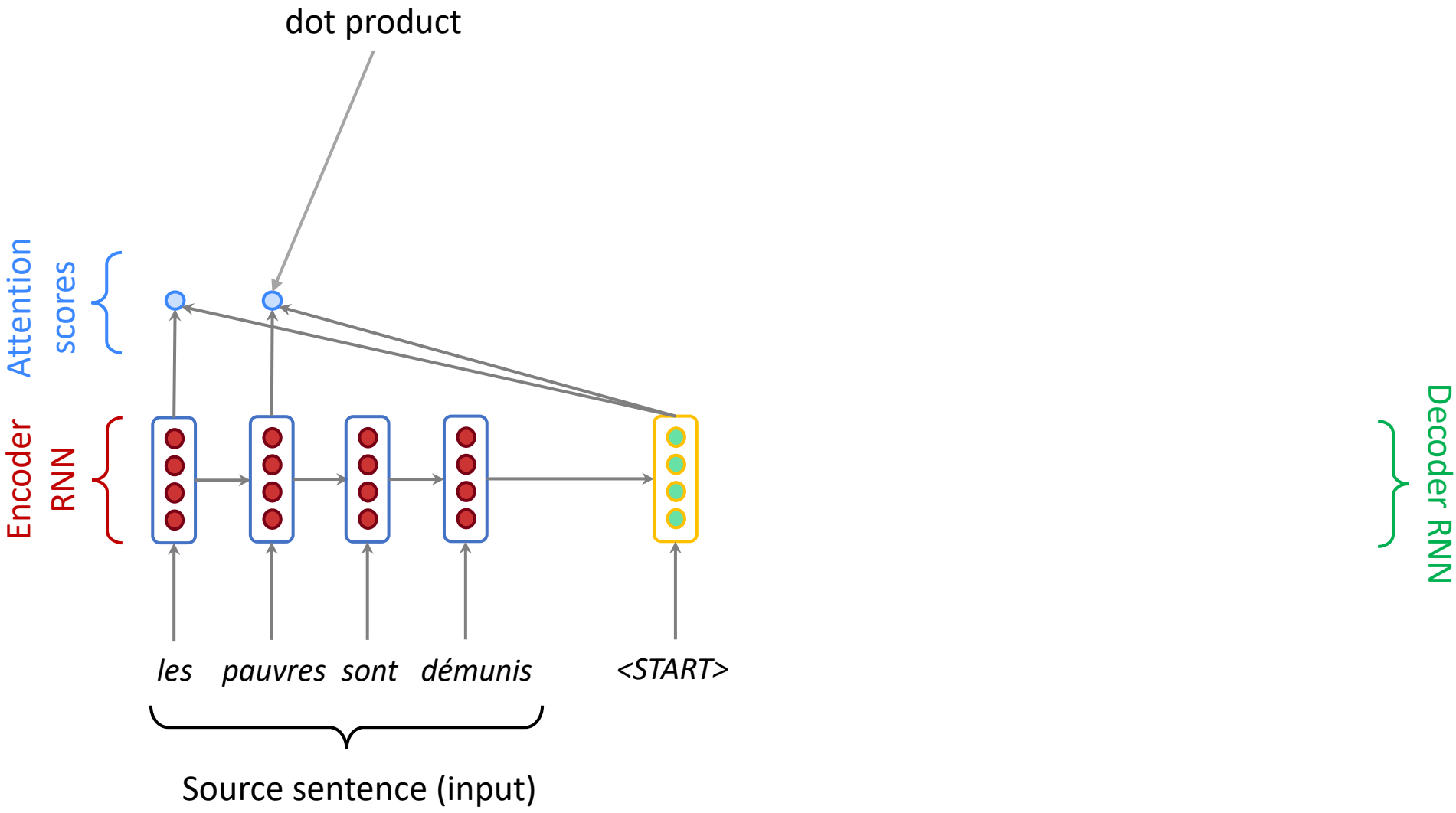
- **Attention** provides a solution to the bottleneck problem.

- Core idea: on each step of the decoder, *focus on a particular part* of the source sequence

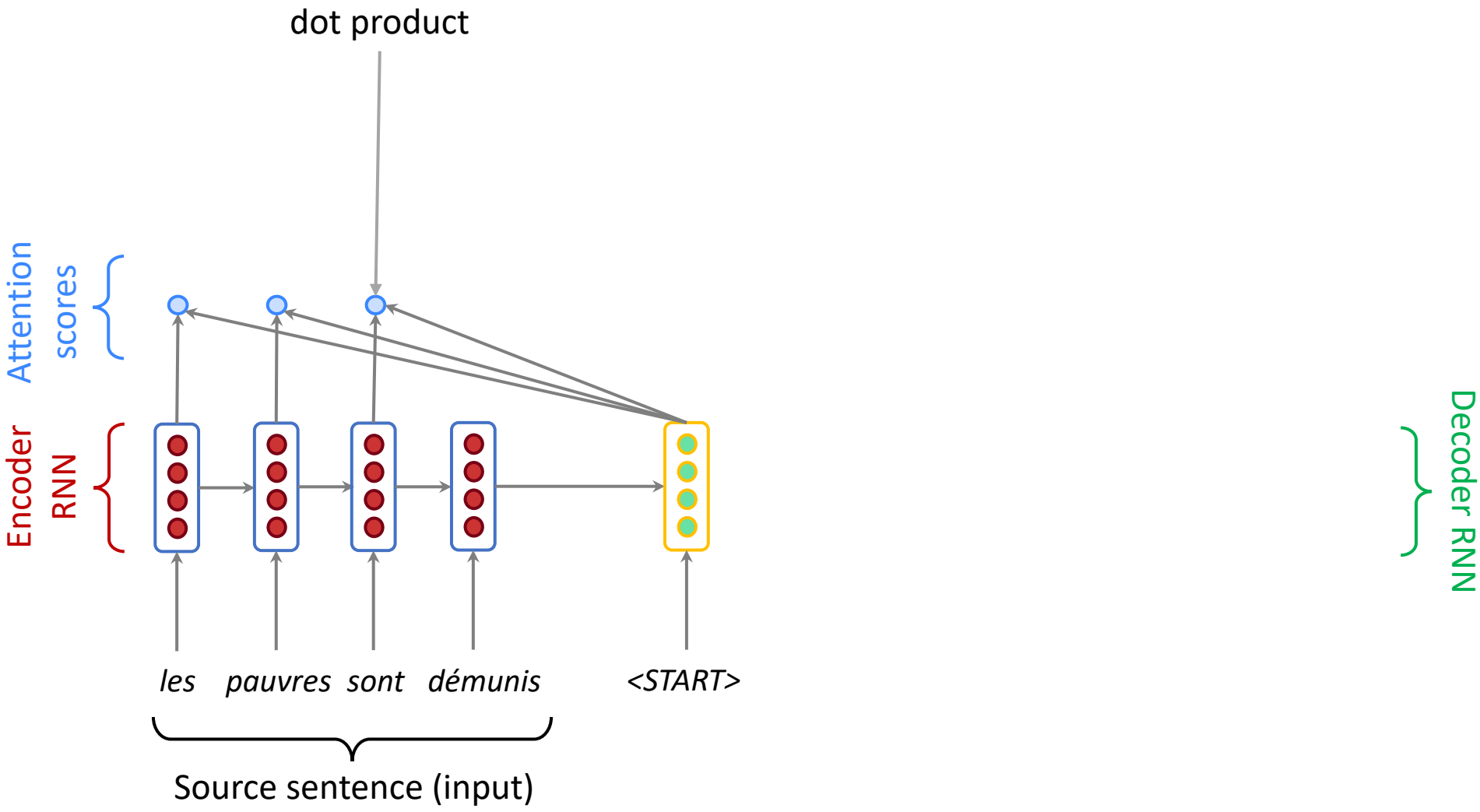
Encoder-Decoder with Attention



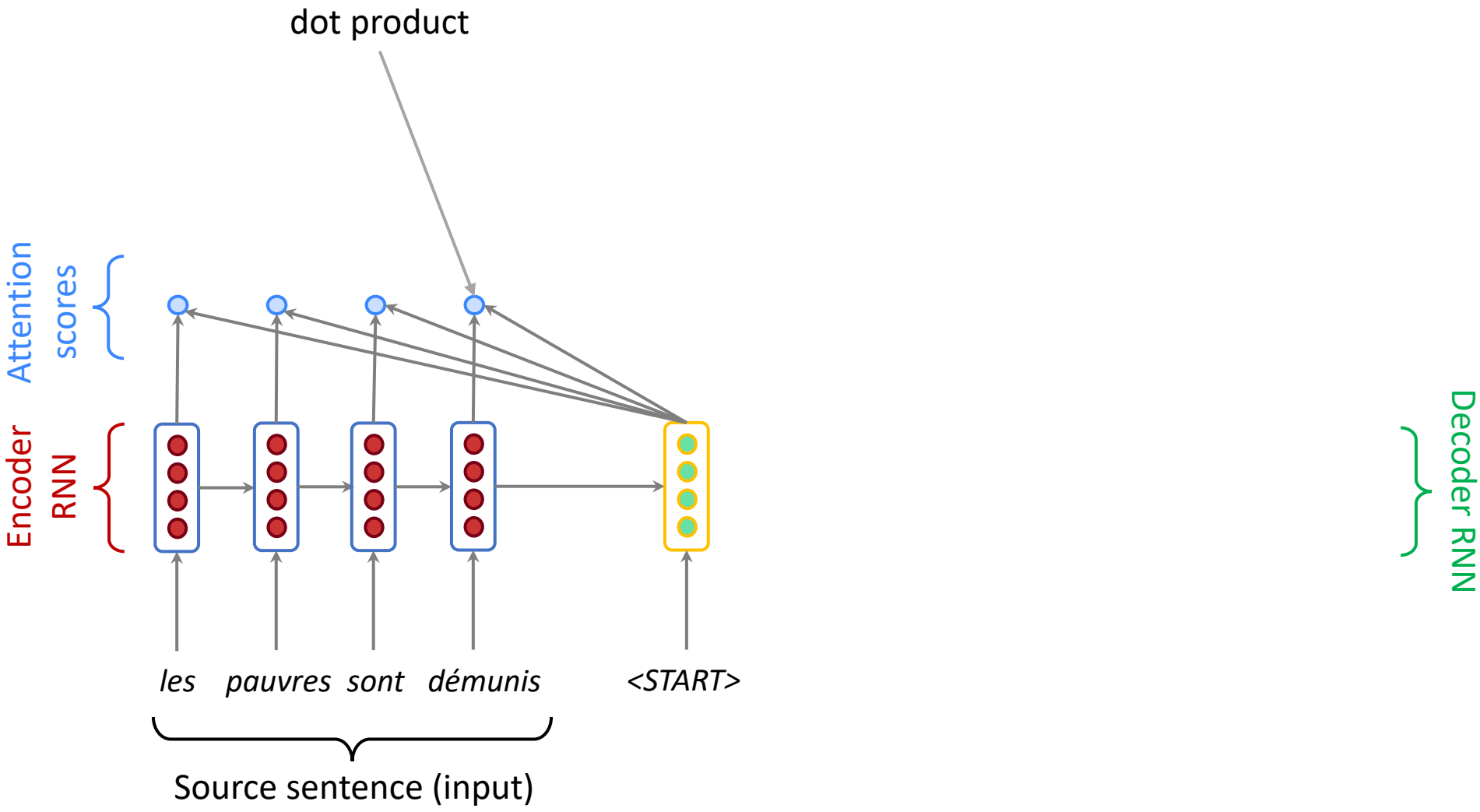
Seq2Seq with attention



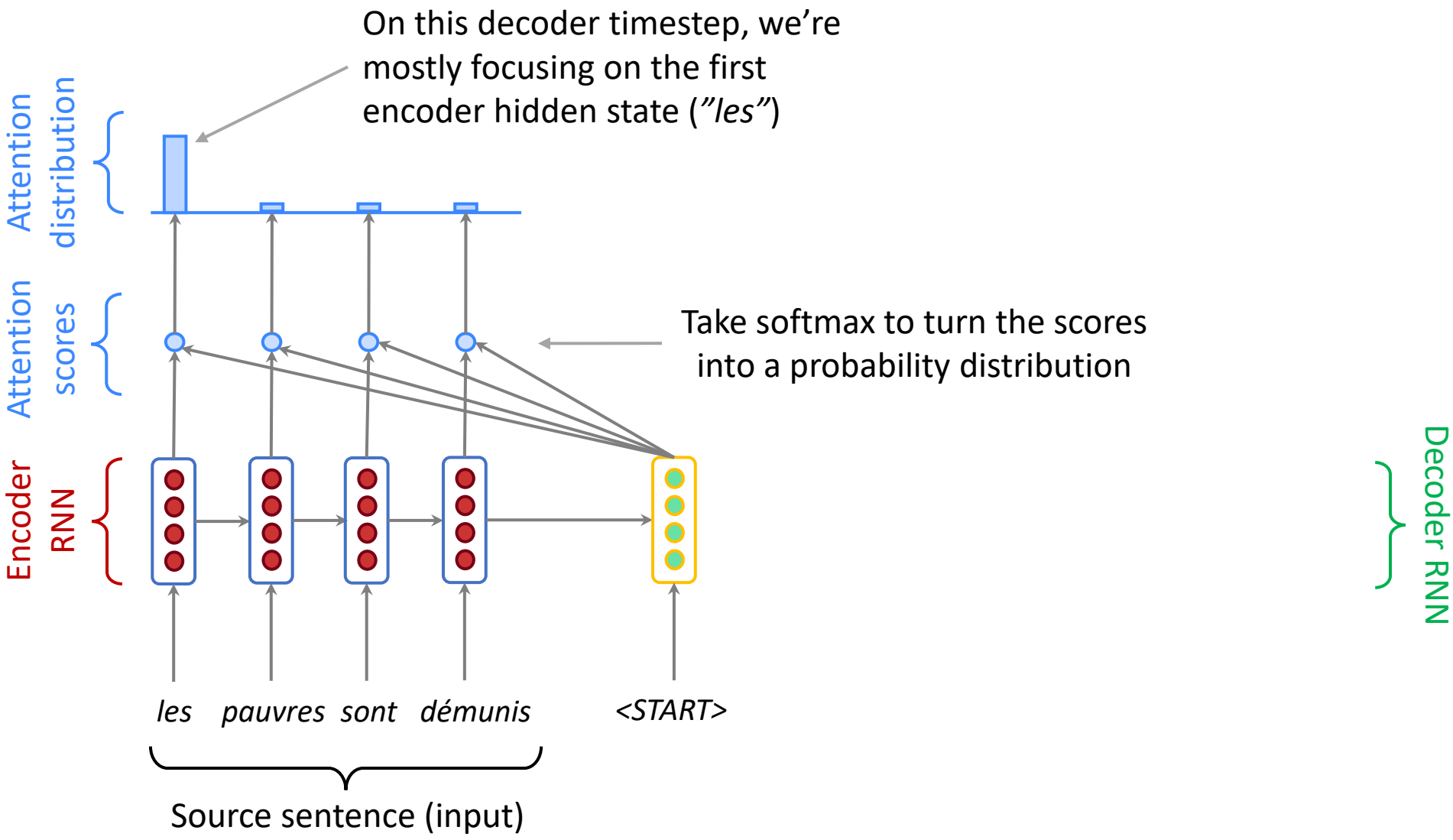
Sequence-to-sequence with attention



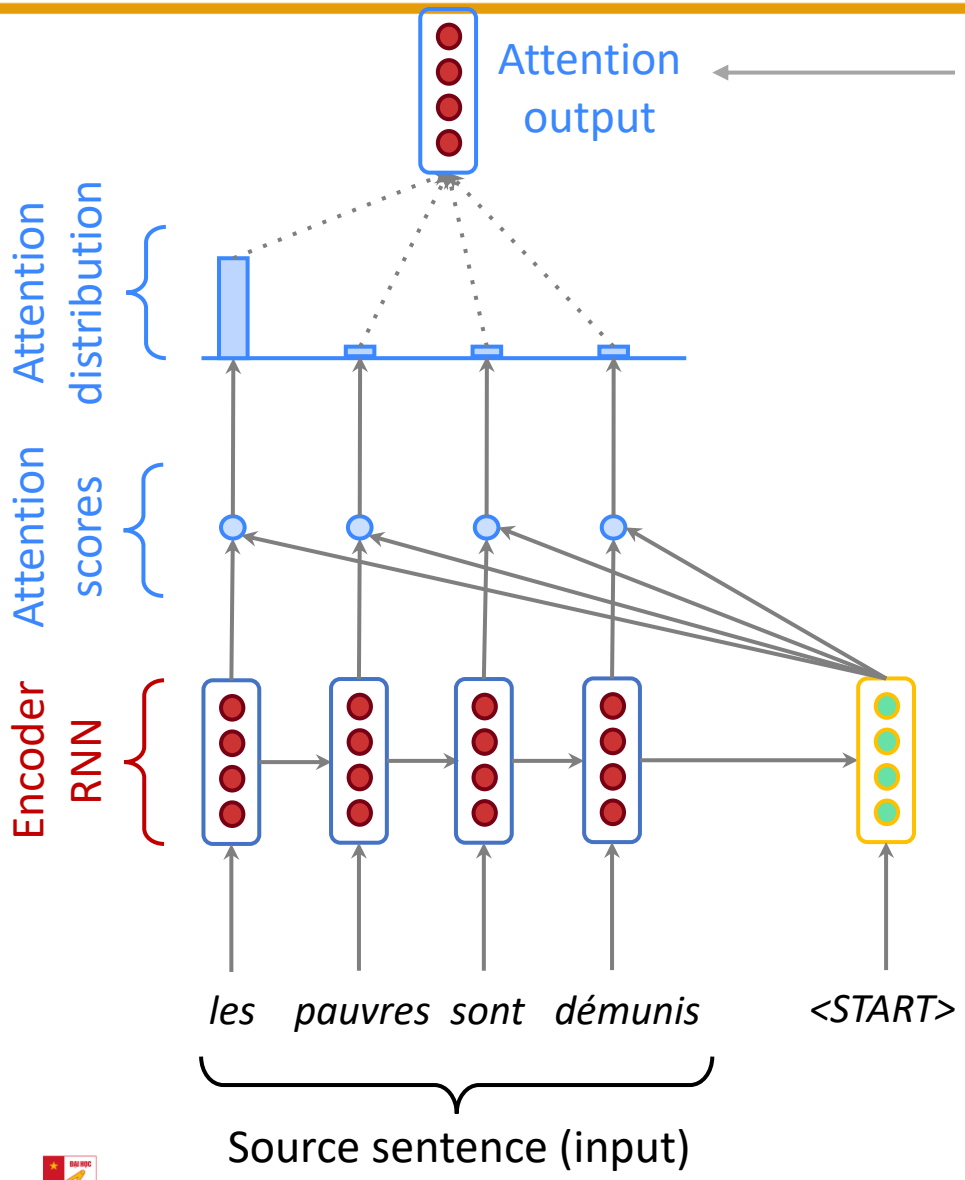
Sequence-to-sequence with attention



Sequence-to-sequence with attention



Sequence-to-sequence with attention

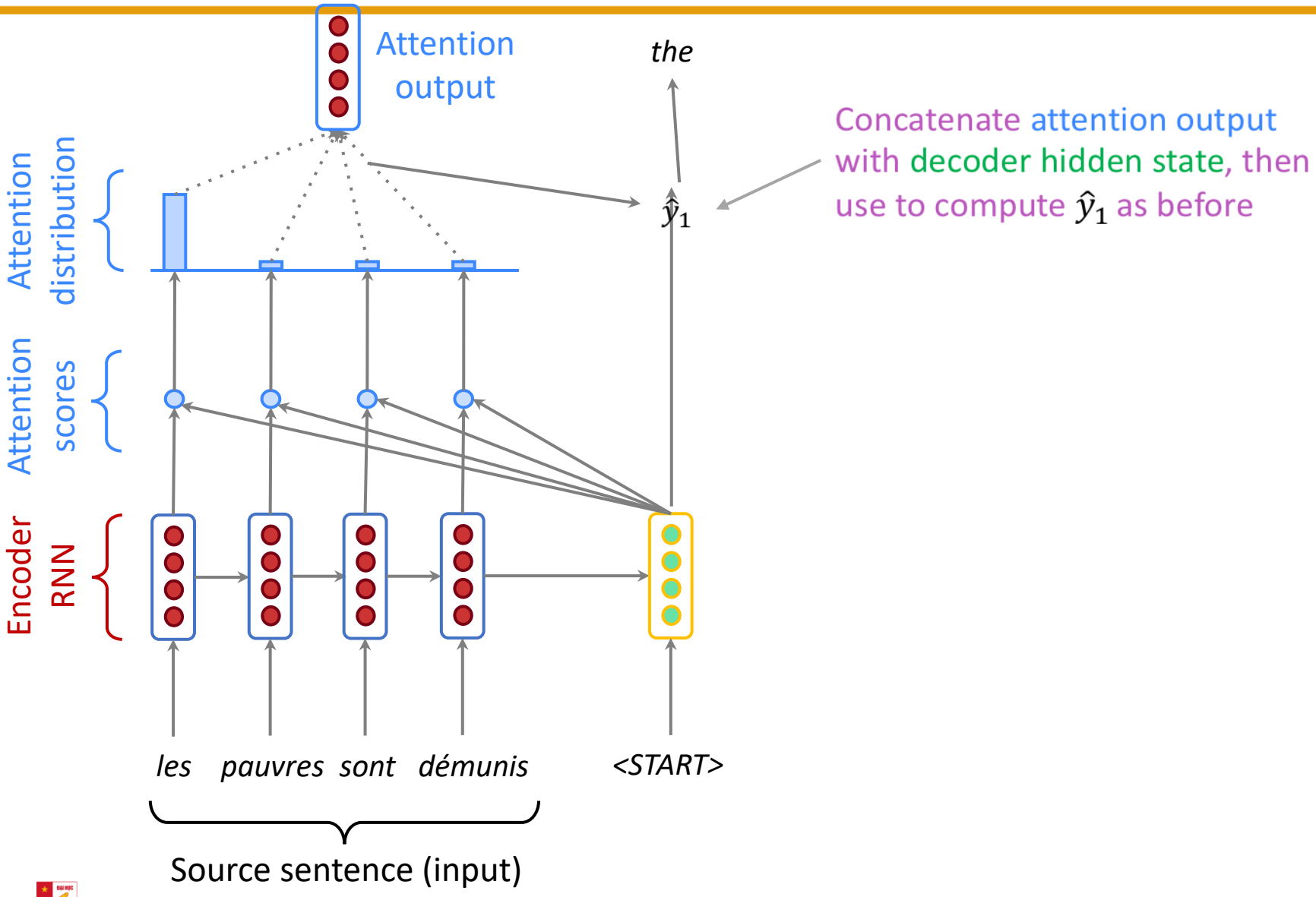


Use the attention distribution to take a weighted sum of the encoder hidden states.

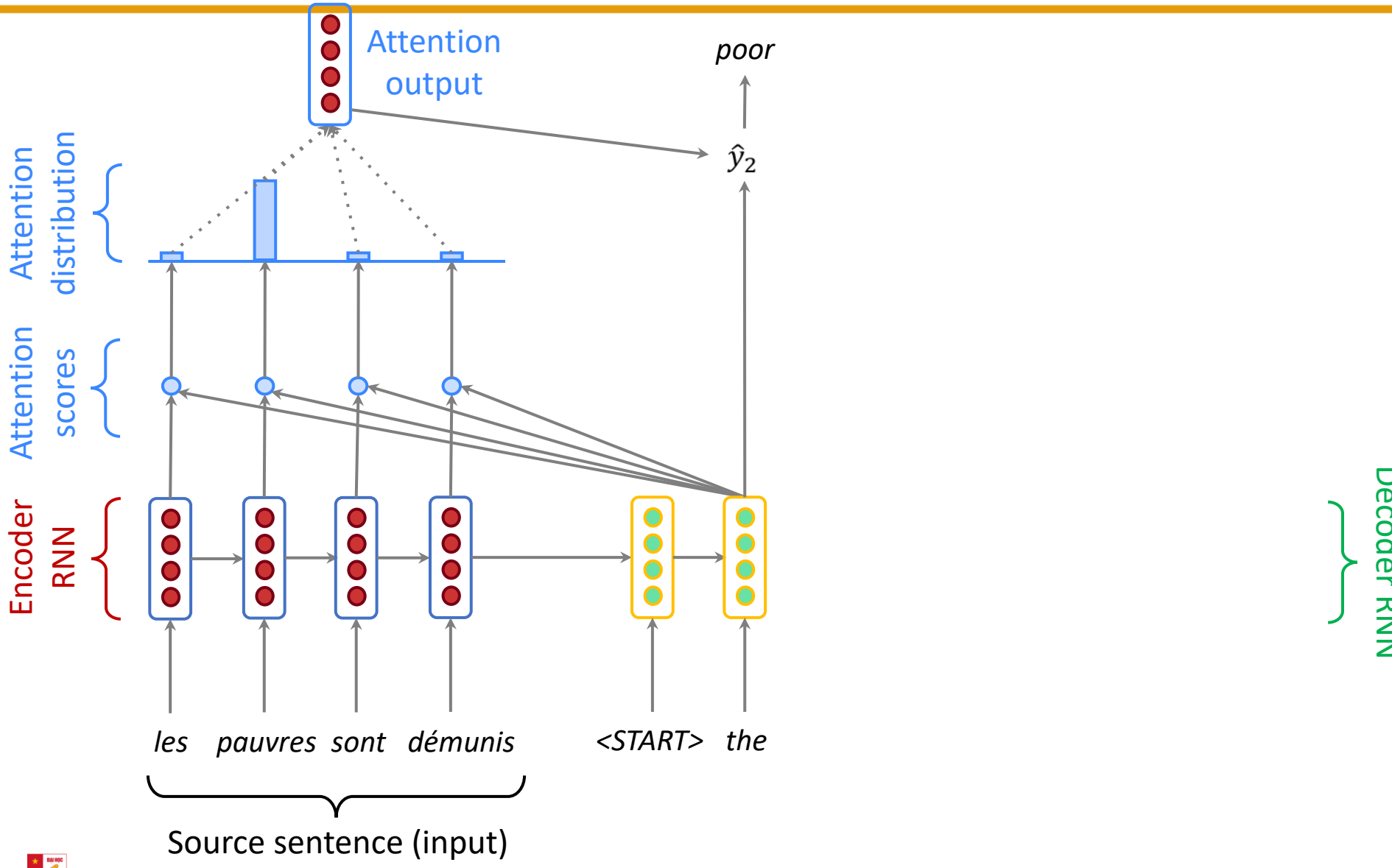
The attention output mostly contains information the hidden states that received high attention.

Decoder RNN

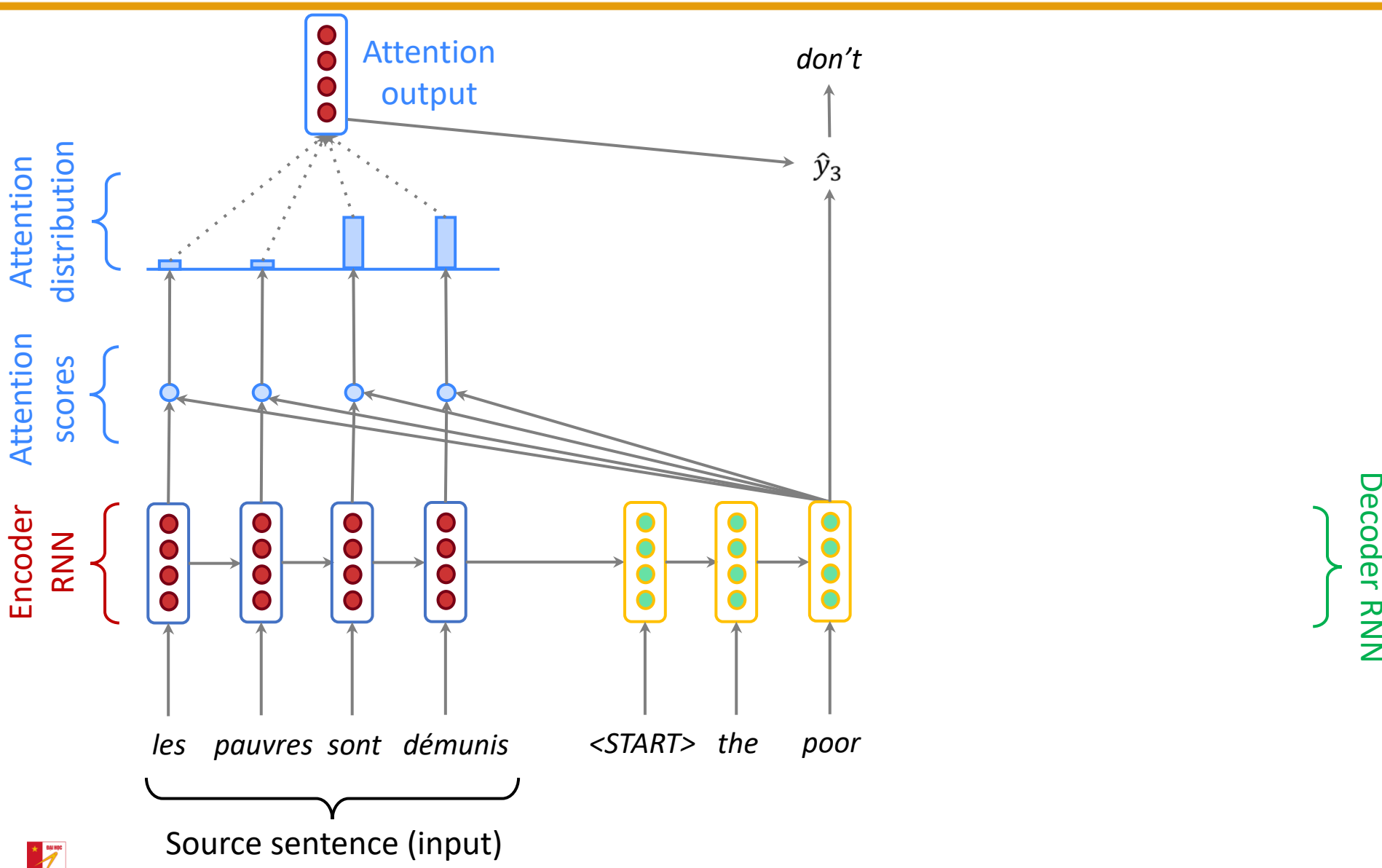
Sequence-to-sequence with attention



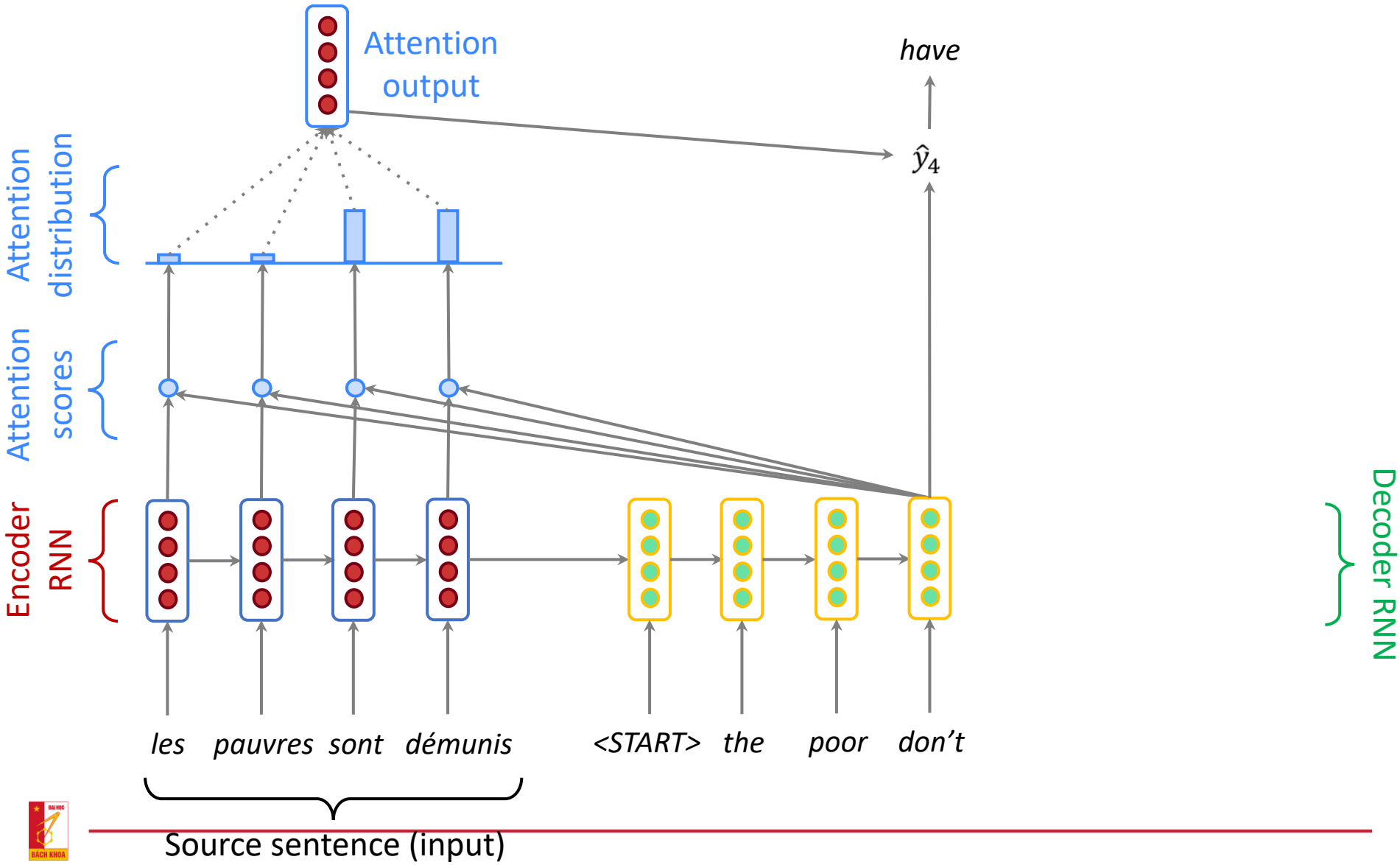
Sequence-to-sequence with attention



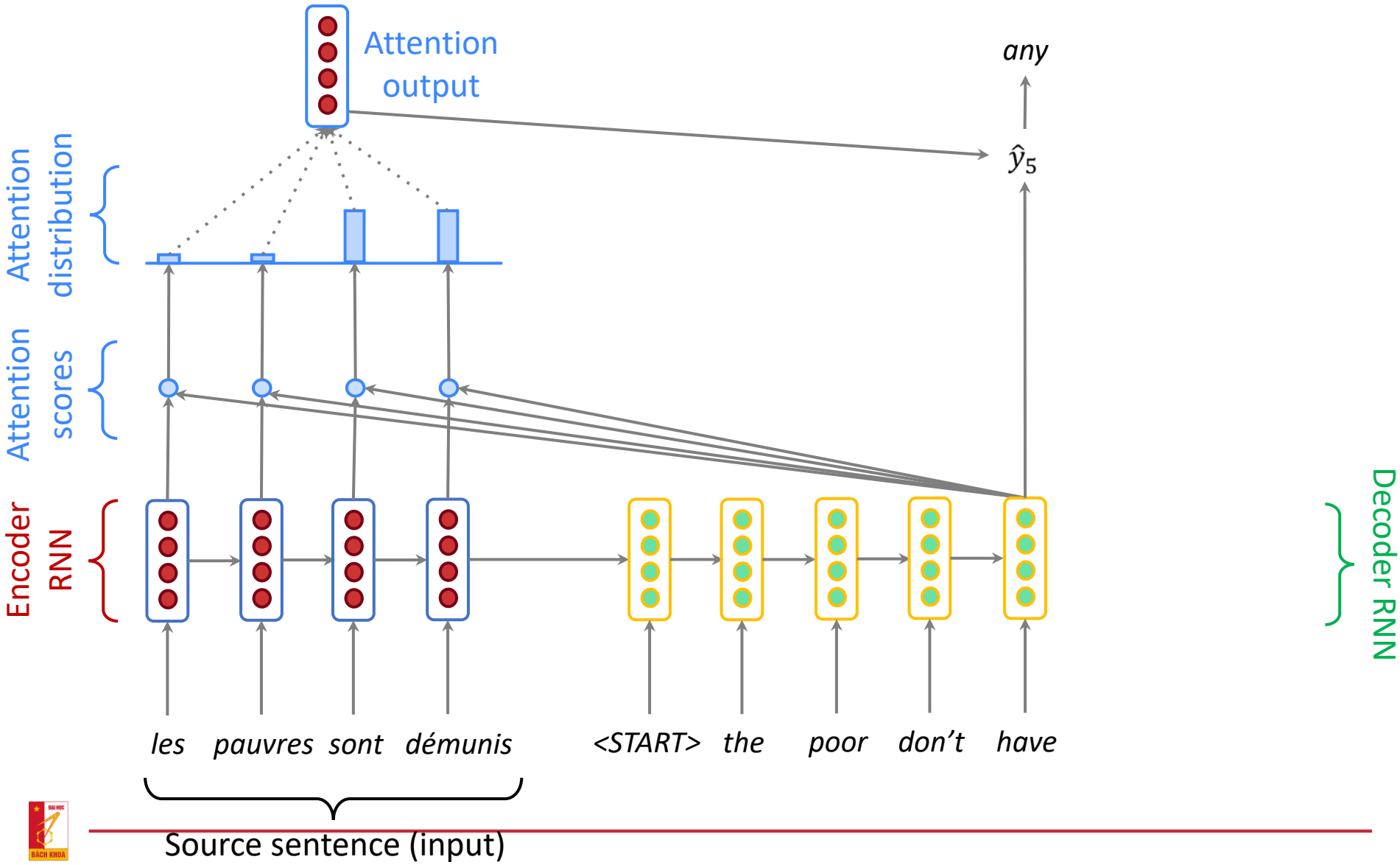
Sequence-to-sequence with attention



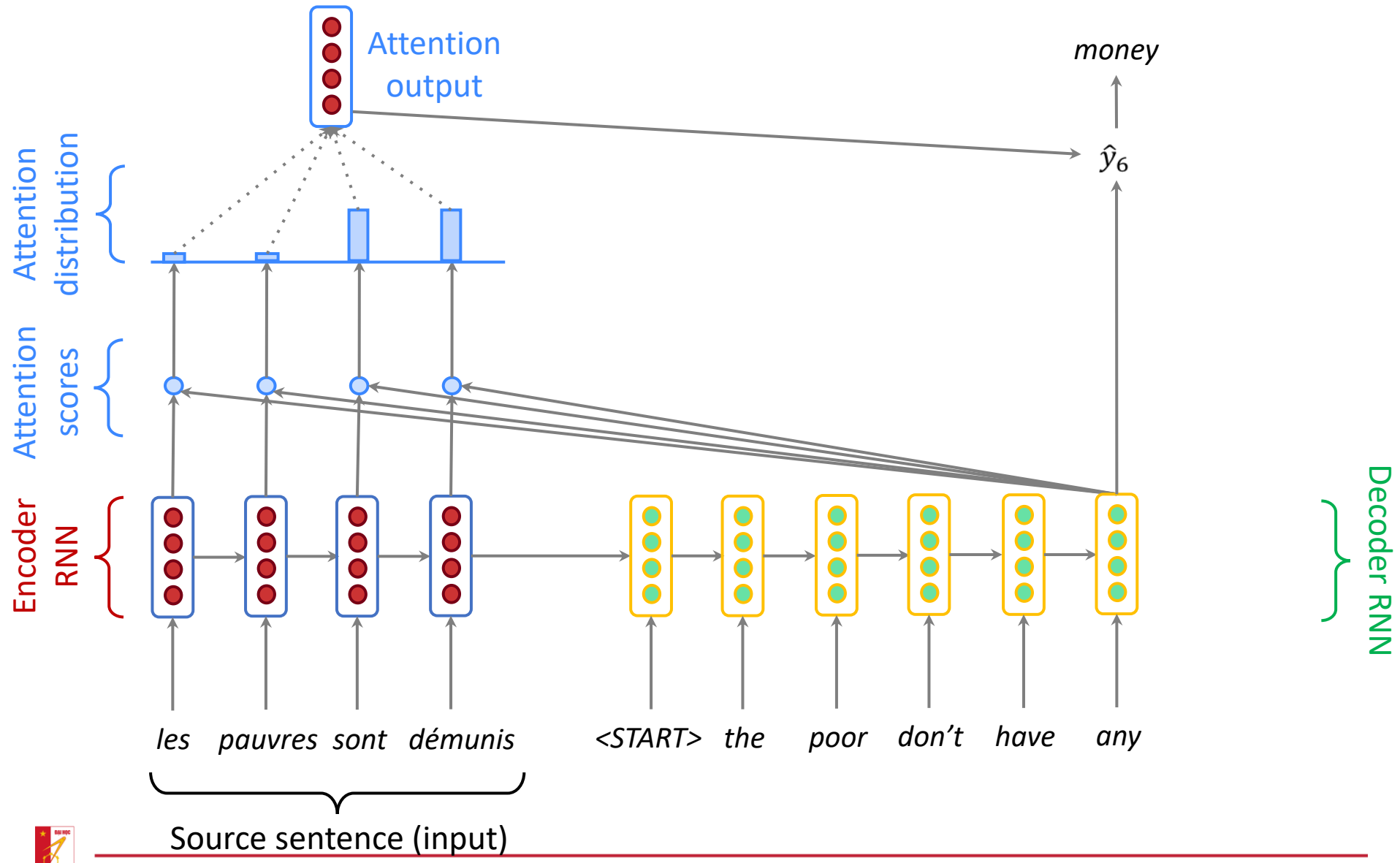
Sequence-to-sequence with attention



Sequence-to-sequence with attention



Sequence-to-sequence with attention



Attention: Formula

- We have encoder hidden states $h_1, \ldots, h_N \in \mathbb{R}^h$
- On timestep *t*, we have decoder hidden state $s_t \in \mathbb{R}^h$
- We get the attention scores e^t for this step:

$$\boldsymbol{e}^t = [\boldsymbol{s}_t^T \boldsymbol{h}_1, \ldots, \boldsymbol{s}_t^T \boldsymbol{h}_N] \in \mathbb{R}^N$$

- We take softmax to get the attention distribution α^t for this step (this is a probability distribution and sums to 1)

$$\alpha^t = \text{softmax}(\boldsymbol{e}^t) \in \mathbb{R}^N$$

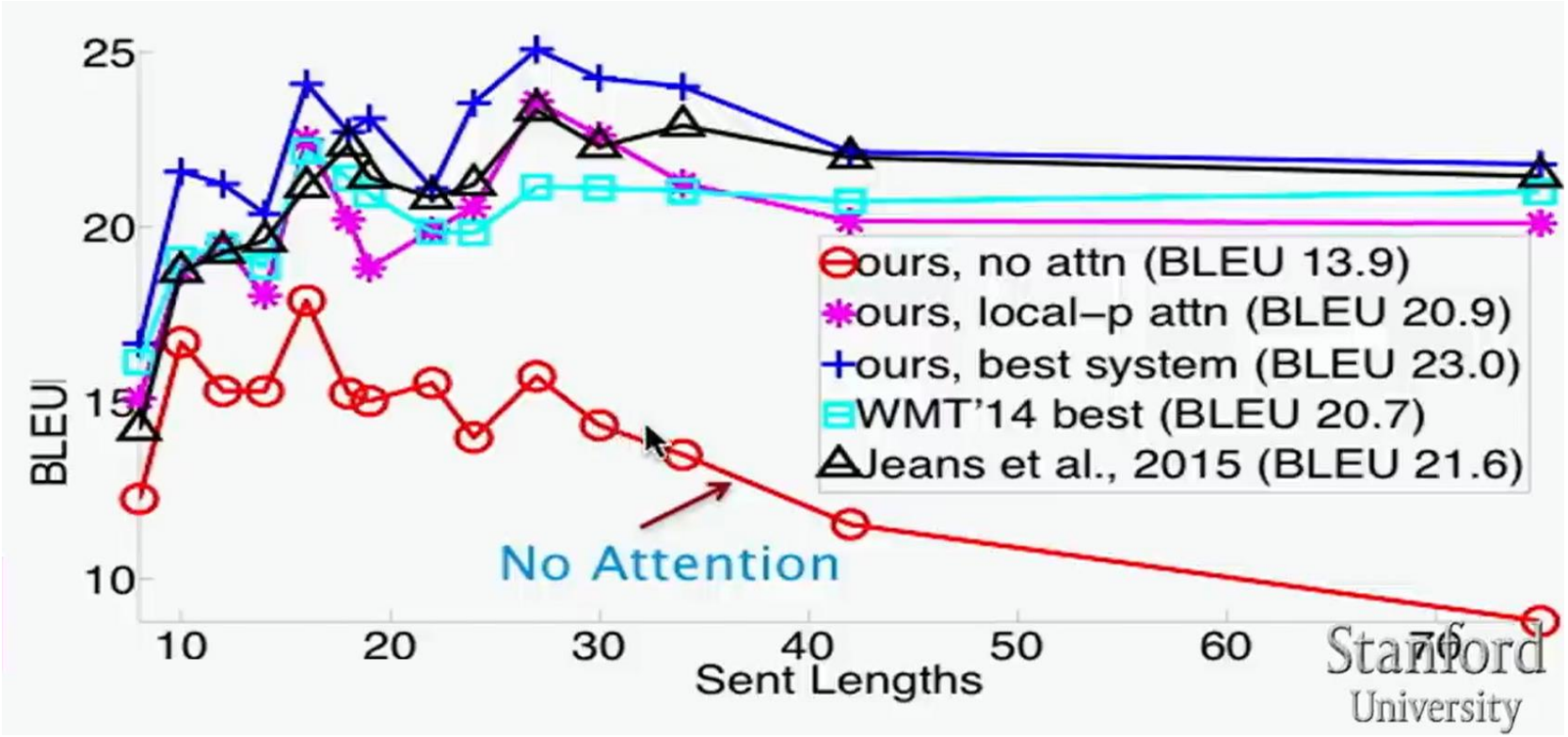
- We use α^t to take a weighted sum of the encoder hidden states to get the attention output $\boldsymbol{a}_t$

$$\boldsymbol{a}_t = \sum_{i=1}^{N} \alpha_i^t \boldsymbol{h}_i \in \mathbb{R}^h$$

- Finally we concatenate the attention output $\boldsymbol{a}_t$ with the decoder hidden state s_t and proceed as in the non-attention seq2seq model

$$[\boldsymbol{a}_t; \boldsymbol{s}_t] \in \mathbb{R}^{2h}$$

Attention translates long sentences well

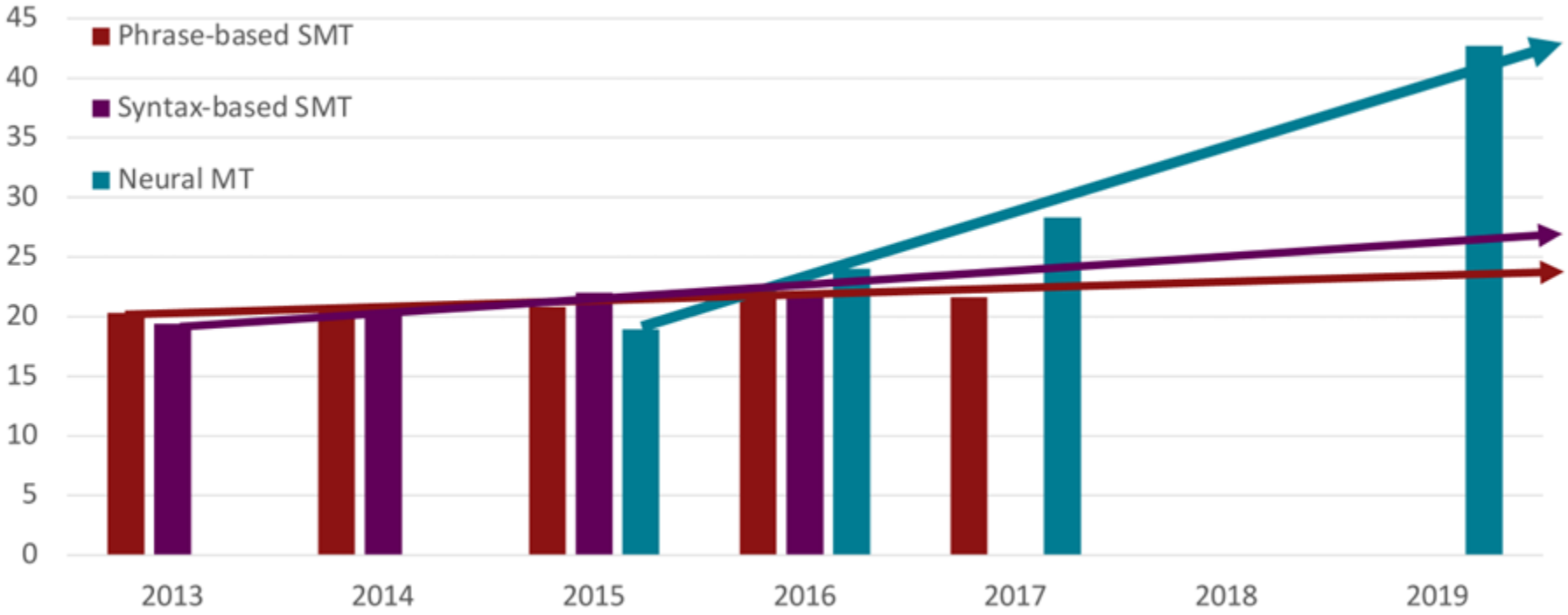


Translate English to German

source	Orlando Bloom and *Miranda Kerr* still love each other
human	Orlando Bloom und **Miranda Kerr** lieben sich noch immer
+attn	Orlando Bloom und **Miranda Kerr** lieben einander noch immer .
base	Orlando Bloom und **Lucas Miranda** lieben einander noch immer .

Development of the MT system over time

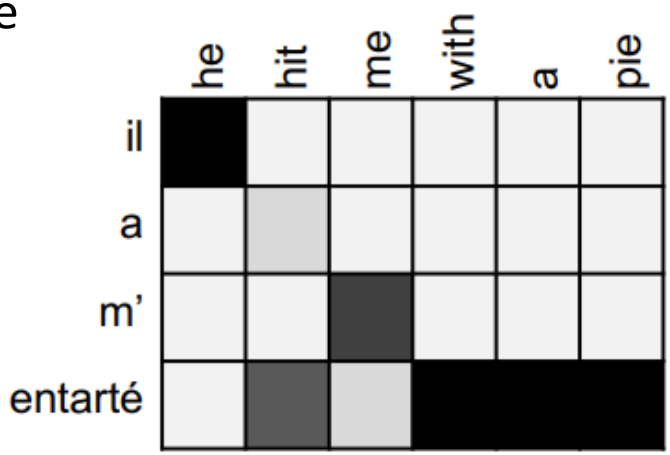
[Edinburgh En-De WMT newstest2013 Cased BLEU; NMT 2015 from U. Montréal; NMT 2019 FAIR on newstest2019]



Source: Neural Machine Translation: Breaking the Performance Plateau (meta-net.eu)

Attention is excellent (Bahdanau at al., 14164 citations)

- Attention significantly improves NMT performance
  - It's very useful to allow decoder to focus on certain parts of the source
- Attention solves the bottleneck problem
  - Attention allows decoder to look directly at source; bypass bottleneck
- Attention helps with vanishing gradient problem
  - Provides shortcut to faraway states
- Attention provides some interpretability
  - By inspecting attention distribution, we can see what the decoder was focusing on
  - We get alignment for free!
  - This is cool because we never explicitly trained an alignment system
  - The network just learned alignment by itself



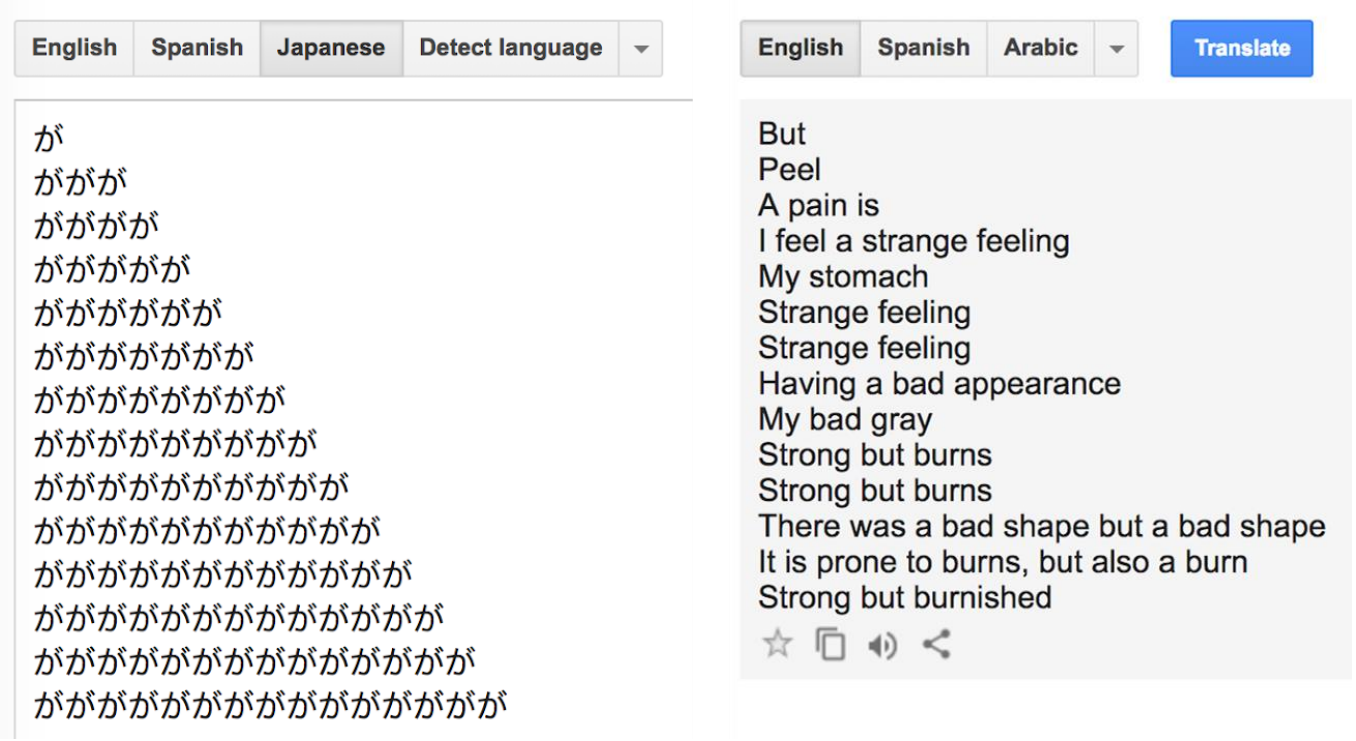
NMT: NLP Deep Learning's biggest success story

Neural Machine Translation went from a fringe research activity in **2014** to the leading standard method in **2016**

- **2014**: First seq2seq paper published
- **2016**: Google Translate switches from SMT to NMT
- This is amazing!
  - **SMT** systems, built by hundreds of engineers over many years, outperformed by NMT systems trained by a handful of engineers in a few months

So has MT been resolved?

- Nope!
- Uninterpretable systems do strange things



Source: http://languagelog.ldc.upenn.edu/nll/?p=35120#more-35120